

TalonG: Bandwidth-Efficient Two-Round Threshold Signatures from Lattices

Liming Gao^{1,2}, Guofeng Tang^{3*}, Dingding Jia^{1,2}, Yijian Liu^{1,2}, Bingqian Liu^{1,2}, Xianhui Lu^{1,2}, Kunpeng Wang^{1,2}, and Yongjian Yin^{1,2}

¹ State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{gaoliming,jiadingding,liuyijian}@iie.ac.cn

{liubingqian,luxianhui,wangkunpeng,yinyongjian}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

³ Singapore Management University, tang.guofeng789@gmail.com

Abstract. Threshold signatures split the secret key among n parties, where any subset of at least t parties can collaboratively produce a valid signature. They have been widely deployed in applications such as blockchain systems. Lattice-based threshold signatures have also attracted considerable attention due to their post-quantum security guarantees. However, existing lattice-based constructions still face significant efficiency challenges, particularly when the number of parties becomes large. Recent lattice-based threshold signatures such as TRaccoon (Eurocrypt’24) and Ringtail (S&P’25) support large thresholds, but they either require three interaction rounds or incur heavy communication costs in the two-round setting, limiting their practicality.

In this paper, we present TalonG, a novel two-round lattice-based threshold signature that overcomes these limitations via a new trapdoor semi-commitment technique. This variant of commitment relaxes the standard binding requirement to a weaker form, allowing an efficient instantiation from the NTRU assumption and enabling a compact two-round signing protocol with low communication.

For $t = 1024$ and 128-bit security, TalonG achieves significant improvements among existing lattice-based threshold signatures: its total communication per party and public key size are both minimal, at 26.9 KB and 2.0 KB, respectively. While the resulting signature size is larger (17.7 KB), it remains practical and highly competitive. TalonG is thus well-suited for real-world large-scale deployments where both round efficiency and communication load are critical.

Keywords: Threshold signatures · Lattice-based signatures · Trapdoor commitments · NTRU-type trapdoor.

* Corresponding Author.

1 Introduction

The concept of threshold signatures was first introduced in [Des90,DF90]. It represents a cryptographic extension that combines digital signatures with threshold secret sharing. In a (t, n) threshold signature scheme, the signing key is distributed among n parties such that any group of at least t honest parties can jointly generate a valid signature without reconstructing the secret key, while any group of fewer than t parties cannot forge a new signature. Threshold signatures strengthen key security by removing single points of failure, making them particularly valuable in both blockchain systems and academic research. Their importance is further underscored by NIST’s standardization of Multi-Party Threshold Cryptography [BP25].

In the classical setting, various works have investigated the threshold version of the standard signatures, such as threshold Schnorr [KG20,NRS21,Lin24], threshold ECDSA [CGG+20,THL+24,TX25], threshold BLS [DR24], and threshold BBS+ [DKL+23]. However, it is now widely recognized that these schemes will be insecure in a quantum world, where classical hard problems such as the discrete logarithm can be efficiently solved by Shor’s algorithm [Sho97] on large-scale quantum computers.

Existing lattice-based threshold signatures. Lattices are considered as a reliable foundation for post-quantum cryptography. Consequently, lattice-based threshold signatures [dPKM+24,EKT24,BKL+25,dPN25] and multi-signatures [DOTT21,BTT22,Che23] have attracted considerable attention from academic research, where a multi-signature is an n -out-of- n full threshold signature.

As a prelude to our work, we first give a brief overview of existing lattice-based threshold signature schemes. For a more detailed discussion, refer to Section 1.2. When evaluating a threshold signature (or multi-signature) scheme, the following metrics are of particular interest.

- *Large-scale support:* It is an important metric since NIST’s multi-party threshold cryptography call [BP25, Table 16] highlights use cases with a large number of parties (e.g., up to 1024). In real-world applications such as blockchain systems, the number of participants can reach hundreds or even thousands, making support for large-scale settings a practical and necessary consideration. Currently, many lattice-based threshold schemes can achieve this, such as [Che23,EKT24,dPKM+24,BKL+25]. However, some schemes, such as [TPCZ23,ADP24,ENP24,CTZ24,GKS24,dPN25], are only suitable for the small-party case (e.g., < 20), as they rely on fully homomorphic encryption, replicated secret sharing, or other heavy techniques.
- *Round complexity:* In wide-area or large-scale deployments, network latency dominates cost. High round complexity often results in significant latency, limiting the practicality of such protocols. Lattice-based threshold signature schemes with the Fiat-Shamir framework can achieve a minimum of two communication rounds, as in [Che23,CTZ24,EKT24,BKL+25].
- *Communication and size efficiency:* Lower communication overhead and smaller signature and public key sizes make lattice-based threshold signa-

tures more suitable for enabling post-quantum migration. Nevertheless, current schemes find it difficult to reconcile scalability with efficiency. For example, while [dPN25] offers compact signatures and low communication cost, it is restricted to small-scale participant settings.

1.1 Our Contributions

We introduce TalonG, a practical two-round lattice-based threshold signature scheme, which supports a large threshold value $t = 1024$. Under the parameter setting corresponding to 128-bit security for $t = 1024$ (see Table 3 for details), the signature size is 17.7 kilobytes (KB), and the public key size is 2.0 KB. The total communication cost per signer is 26.9 KB.

Technically, our construction builds on Raccoon [dPKPR24]—a signature scheme that avoids rejection sampling and is derived from Lyubashevsky’s lattice-based Fiat-Shamir framework [Lyu09, Lyu12]. Existing threshold signatures face a trade-off: they either require three rounds or incur high communication costs in the two-round setting. To overcome this limitation, we propose a new trapdoor semi-commitment scheme. This primitive only requires a weak form of binding—rather than the standard binding property used in prior threshold schemes such as [DOTT21, GKS24]. We demonstrate that the loss of the standard binding property does not compromise security, since any adversarial advantage can be reduced to the hardness of the Ring Short Integer Solution (RSIS) problem. By replacing the standard commitment with our semi-commitment and instantiating the semi-commitment with an efficient NTRU-type trapdoor, we significantly reduce communication overhead and signature size while preserving two-round execution. This technique is generic and can also be applied to improve the efficiency of other trapdoor-commitment-based schemes like [DOTT21, GKS24]. Furthermore, we employ modulus switching to achieve additional compression, resulting in even more compact signatures.

A trade-off is that both rounds of our protocol must be executed online, whereas one round in [EKT24] and [BKL⁺25] can be pre-computed offline.

We compare the prior lattice-based threshold signatures with ours. As shown in Table 1, under large-scale settings, our scheme attains minimal communication and public key sizes, showing a significant improvement over prior works, while maintaining the signature size comparable to those of existing schemes. Concretely, we offer the following advantages:

1. **Low Communication Overhead.** We propose a practical threshold signature scheme that requires only two rounds. Moreover, unlike prior works [EKT24, BKL⁺25] where each signer must transmit multiple polynomials as “commitment” messages in the first round, TalonG requires only a single polynomial per signer in the first round when instantiated with our semi-commitment scheme. This design significantly reduces communication overhead and lowers interaction latency.
2. **Compact Verification Key.** We introduce a novel parameter analysis for public key compression and verification bounds. Unlike other threshold sig-

Table 1: **The comparisons of lattice-based threshold signatures.** $|\text{trans}|/t$ denotes the communication cost (KB) for each party; $|\text{vk}|$ and $|\text{sig}|$ represent the sizes (KB) of the public key and the signature, respectively. $t \leq n$ indicates that the scheme supports arbitrary threshold structures, while $t = n$ means it is an n -out-of- n threshold signature or multi-signature.

Scheme	t	Rounds	$ \text{trans} /t$	$ \text{vk} $	$ \text{sig} $	Types	Assumptions
[ADP24]	7	3	25	7	12	$t = n$	MLWE, MSIS
[ENP24]	16	4	364	12.8	12.3	$t \leq n$	RLWE, RSIS
[CTZ24]	5	2	1526	47	258	$t \leq n$	MSIS
[GKS24]	5	2	3072	13	46	$t \leq n$	RLWE, RSIS
[dPN25]	8	3	7.5	2.6	2.7	$t \leq n$	MLWE, MSIS
[dPKM ⁺ 24]	1024	3	40.8	3.9	12.7	$t \leq n$	MLWE, MSIS
[Che23]	1024	2	300	10.5	30.9	$t = n$	MLWE, MSIS
[EKT24]	1024	2	276.1	5.5	10.8	$t \leq n$	AOM-MLWE, MSIS
[BKL ⁺ 25]	1024	2	612.8	4.5	13.4	$t \leq n$	MLWE, MSIS
Ours	1024	2	26.9	2.0	17.7	$t \leq n$	RLWE, RSIS, NTRU

nature schemes that employ modulus rounding [dPKM⁺24], our construction uses modulus switching to allow more bit compression. By maximizing the number of compressed bits while preserving correctness and security, we achieve the smallest verification key size among existing lattice-based threshold signatures.

3. **High Computational Efficiency.** As demonstrated in Table 4, our scheme achieves substantial computational improvements over TRaccoon [dPKM⁺24] and Ringtail [BKL⁺25], particularly in the signing phase. For example, at $t = 1024$, the total signing time of our scheme is only 140.713 ms—approximately 18 times faster than Ringtail’s 2487.886 ms on the identical processor. The primary technical driver behind this significant performance improvement is our design, which substantially reduces the number of polynomial operations.

1.2 Related Works

Several lattice-based threshold signature schemes have advanced the state of the art in recent years. Boneh et al. [BGG⁺18] used fully homomorphic encryption (FHE) to create a universal thresholdizer. Applying it to lattice signatures yields a lattice-based threshold signature scheme. Agrawal et al. [ASY22] later refined this approach using Rényi divergence techniques to reduce FHE noise parameters. These two schemes, while round-optimal, remain theoretical and are difficult to instantiate due to their high circuit computational complexity. Taking advantage of the Fiat–Shamir paradigm, a variety of more efficient lattice-based threshold signature schemes have been developed.

Lattice-based Multi-Signatures. A notable step toward practicality was made by Damgård et al. [DOTT21], who constructed a two-round multi-signature

without relying on FHE. The scheme employed the Fiat-Shamir with Aborts (FSwA) paradigm and incorporated a lattice-based homomorphic trapdoor commitment (HTDC). Nevertheless, this scheme was limited to the n -out-of- n setting and did not support an arbitrary threshold t .

Chen [Che23] proposed DualMS, an efficient lattice-based two-round multi-signature scheme that significantly reduces both signature and public key sizes compared to prior works like MuSig-L [BTT22]. For 128-bit security, DualMS achieves approximately 27 KB (public key + signature) for up to 32 signers and 41 KB for up to 1024 signers. It builds on the idea from the Okamoto signature scheme [Oka92], with the core innovation being a trapdoor-free “dual signing simulation” technique. DualMS avoids the lattice trapdoors used in [DOTT21] and MuSig-L, thereby improving efficiency. On the other hand, its extension to threshold signatures remains unclear.

Threshold signatures with small-scale parties. Building upon [DOTT21], Gur et al. [GKS24] incorporated threshold FHE with HTDC to achieve a two-round t -out-of- n threshold signature scheme. By leveraging Fiat-Shamir-type signatures, they reduced circuit computational complexity and provided a concretely instantiable scheme. However, the reliance on HTDC came at the cost of larger signature sizes—for instance, signatures reach 46 KB and verification keys 13 KB in a 3-out-of-5 setting.

Alkadri et al. [ADP24] presented a three-round n -out-of- n threshold signature scheme. Technically, it removes the need for homomorphic commitments and adopts a tree-of-commitments approach to reduce restart probability. For $n = 7$ and 128-bit security, the scheme achieves a public key size of 7 KB and a signature size of 12 KB. However, its computational complexity limits practicality to a small number of signers.

Espitau et al. [ENP24] proposed Pelican, a robust threshold signature scheme based on standard lattice assumptions, which constitutes the first hash-and-sign lattice-based threshold signature. For a threshold of $t = 16$, Pelican produces signatures of about 12.3 KB and public keys of 12.8 KB at the 128-bit security level. The scheme guarantees robustness during both key generation and signing, but its communication overhead grows quadratically with the number of parties. In particular, when $t = 1024$, the communication cost reaches 116 MB, which hinders scalability.

Del Pino et al. [dPN25] proposed a highly compact threshold signature scheme with signatures nearly as small as a single Dilithium signature (2.7 KB) with verification keys of 2.6 KB for small thresholds ($t \leq 8$). Their construction relied on a novel secret sharing scheme with small shares. By improving the analysis of rejection sampling, they achieved tight security under standard MLWE and SelfTargetMSIS assumptions. This scheme significantly narrows the gap between threshold and non-threshold signature sizes, offering a practical solution for small-group settings. Recently, [BCdP⁺25] leverages a more efficient secret-sharing scheme to raise the threshold limit from $t \leq 8$ to $t \leq 64$.

As an alternative approach, Tang et al. [TPCZ23] presented a lattice-based threshold signature using generic MPC techniques, achieving functional inter-

changeability without altering the original parameters. Consequently, the resulting signatures retain the same size as the standard signatures. However, the communication overhead is substantial, making this approach suitable only for a small number of parties.

Threshold signatures with large-scale parties. Del Pino et al. [dPKM⁺24] presented a three-round scheme TRaccoon based on Hint-MLWE and SelfTarget-MSIS assumptions. This construction achieved signatures of about 12.7 KB and verification keys of 3.9 KB for thresholds t up to 1024 without heavy primitives such as FHE/HTDC. Note that for appropriately chosen parameters, the Hint-MLWE problem is equivalent to the standard MLWE problem, and similarly, the SelfTarget-MSIS problem is equivalent to the standard MSIS problem. Therefore, we did not make any special distinction in Table 1. Recently, Katsumata et al. [KRT24] achieved adaptive security on TRaccoon with five rounds of communication, and del Pino et al. [dPKN⁺25] incorporated identifiable abort into it at the cost of higher communication overhead.

Esposito et al. [EKT24] later introduced the first two-round scheme without FHE/HTDC components, relying on the Algebraic One-More MLWE (AOM-MLWE) assumption. Their offline-online design enabled non-interactive signing after preprocessing, yielding signatures of 10.8 KB and verification keys of 5.5 KB for threshold t up to 1024. Zhu and Tessaro [ZT25] subsequently established the security proof of [EKT24] under the Algebraic One-More Module-SIS (AOM-MSIS) assumption, which is equivalent to the standard MSIS and MLWE assumptions; however, this proof applies only in large-parameter regimes, resulting in substantially increased overhead.

Ringtail [BKL⁺25] synthesized these advances through a two-round design based on the standard MLWE assumption. By combining offline preprocessing with efficient online signing, it maintained low signature sizes (13.4 KB) and verification keys (4.5 KB) for 1024 threshold while avoiding non-standard assumptions. This construction represented a state-of-the-art balance between theoretical security, round efficiency, and practical overhead.

Overall, the trajectory of lattice-based threshold signatures has evolved from theoretical frameworks relying on heavy primitives toward efficient and compact signatures under standard assumptions.

2 Technical Overview

Our construction builds upon the lattice-based Fiat-Shamir paradigm. In this section, we review this paradigm, outline the difficulties involved in designing a two-round threshold signature, and summarize how we overcome them.

Lattice-based Fiat-Shamir Signatures. We start from the Raccoon signature scheme [dPKPR24], whose main difference from the Fiat-Shamir-with-Abort signature scheme Dilithium [DKL⁺18] lies in avoiding rejection sampling according to the Hint-MLWE assumption, at the cost of a relatively larger signature size. That is, Raccoon also follows lattice-based “commit-challenge-

response” framework [Lyu09,Lyu12] over a polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. Let $\mathbf{A} \in \mathcal{R}_q^{k \times l}$ be a public matrix, and $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \in \mathcal{R}_q^k$ be a public vector, where the secret vectors $(\mathbf{s}, \mathbf{e}) \in \mathcal{R}_q^l \times \mathcal{R}_q^k$ are sampled from a bounded distribution: typically a discrete Gaussian or a small-norm uniform distribution. To sign a message msg , the signing process proceeds as follows:

1. Sample ephemeral randomness $(\mathbf{y}, \mathbf{e}')$ from a suitable distribution (e.g., discrete Gaussian or bounded uniform). Compute the “commitment” component $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}'$.
2. Compute the challenge $c = \text{H}(\mathbf{w} \parallel \text{msg} \parallel \text{vk})$, where $c \in \mathcal{R}_q$ has small coefficients.
3. Compute the response $(\mathbf{z}, \Delta) = (\mathbf{y} + \mathbf{s} \cdot c, \mathbf{e}' + \mathbf{e} \cdot c)$.

Finally, the verification checks if $(\mathbf{z}, \Delta)$ are short and $c = \text{H}(\mathbf{A}\mathbf{z} - \mathbf{b} \cdot c + \Delta \parallel \text{msg} \parallel \text{vk})$. There are several ways to ensure that $(\mathbf{z}, \Delta)$ does not leak $(\mathbf{s}, \mathbf{e})$ ’s information:

- (i) rejection sampling [Lyu09,Lyu12,DKL⁺18];
- (ii) noise flooding [GKPV10,ASY22,CTZ24];
- (iii) the Hint-MLWE assumption [KLSS23,dPKM⁺24,BKL⁺25].

Raccoon uses the Hint-MLWE assumption, which says that the MLWE problem of $(\mathbf{A}, \mathbf{b})$ is still hard with the knowledge of many hints $(\mathbf{z}, \Delta)$.

Three-Round Threshold Signatures. The transformation to a three-round threshold signing protocol [dPKM⁺24,KRT24] can be done without relying on any heavy tools and thus naturally supports a large number of participants. The signing algorithms proceed as follows.

Assume that the secret $\mathbf{s}$ is shared among n signers with Shamir’s secret sharing scheme. That is, $\mathbf{s}$ is splitted into $\{\mathbf{s}_i\}_{i \in [n]}$ such that $\mathbf{s} = \sum_{i \in S} \lambda_i \mathbf{s}_i$ for any $|S| = t$, $S \subseteq [n]$, where λ_i is the i th Lagrange coefficient. Signer j ’s public key share is $\mathbf{b}_j = \mathbf{A}\mathbf{s}_j + \mathbf{e}_j$. The three-round signing protocol proceeds as follows.

- **Round 1:** Each signer j samples ephemeral randomness $(\mathbf{y}_j, \mathbf{e}'_j)$ from a suitable distribution and computes the commitment share $\mathbf{w}_j = \mathbf{A}\mathbf{y}_j + \mathbf{e}'_j$. Broadcast $\text{H}_0(\mathbf{w}_j)$ with $\text{H}_0(\cdot)$ being a random oracle.
- **Round 2:** Broadcast $\mathbf{w}_j$ and run a view-authentication process to ensure that honest parties have the identical view of the first-round messages.
- **Round 3:** Each signer j computes the challenge $c = \text{H}(\sum_{i \in S} \mathbf{w}_i \parallel \text{msg} \parallel \text{vk})$ and the response share $(\mathbf{z}_j, \Delta_j) = (\mathbf{y}_j + c\lambda_j \mathbf{s}_j + \mathbf{m}_j, \mathbf{e}'_j + c\mathbf{e}_j)$ where $\mathbf{m}_j$ ’s are random shares of zero. Broadcast $(\mathbf{z}_j, \Delta_j)$.

The sum of any t or more than t shares $(\mathbf{z}_j, \Delta_j)$ is a Raccoon signature $(\mathbf{z}, \Delta)$. As the secret share $\mathbf{s}_j$ is amplified by the large Lagrange coefficient λ_j , here the zero’s random share $\mathbf{m}_j$ is employed to hide the secret shares.

Improve to Two-Round. It should be emphasized at the outset that merely removing the first round of transmitting $\text{H}_0(\mathbf{w}_j)$ would compromise security. This would allow the adversary to manipulate $\mathbf{w}$ to mount attacks such as the concurrent attack originally described by Drijvers et al. [DEF⁺19]. Concretely, the unforgeability of Fiat-Shamir-type signature schemes is proved in

two steps. First, unforgeability under the chosen message attack (CMA) is reduced to unforgeability under no message attack (NMA). Second, unforgeability under NMA is reduced to the underlying hardness assumption. The first step can be proved by invoking honest-verifier zero-knowledge (HVZK) to simulate the signature $(c, \mathbf{z}, \Delta)$, and by programming the random oracle to output c on input $\mathbf{w} = \mathbf{A}\mathbf{z} - \mathbf{b} \cdot c + \Delta$. When adapted to the threshold scenario, it indicates that $\mathbf{w}$ should not be generated before c for the programming to succeed. For this reason, in the first round, $\mathbf{w}_j$ could not be transmitted explicitly.

Nevertheless, the above argument does not imply that a two-round protocol is impossible. Similar to threshold Schnorr, lattice-based threshold signatures can also be realized in two rounds using suitable techniques.

A reliable approach, employed in [EKT24, BKL⁺25], utilizes a technique that requires multiple $\mathbf{w}$ values and then linearly combines them into $\tilde{\mathbf{w}}$, which replaces $\mathbf{w}$ to act as the hash input to generate c . A similar approach has also appeared in the threshold Schnorr scheme [NRS21] and lattice-based multi-signature [BTT22]. Concretely, each signer j generates multiple values $\{\mathbf{w}_{j,i}\}_{i \in [k]}$ in the first round. Then, a random oracle $H_\beta(\cdot)$ is employed to generate coefficients $\{\beta_i\}_{i \in [k]} = H_\beta(\{\mathbf{w}_{j,i}\}_{j \in S, i \in [k]})$, where S is the signer set. These coefficients are used to form the linear combination $\tilde{\mathbf{w}}_j = \sum_{i \in [k]} \beta_i \mathbf{w}_{j,i}$. Finally, the combined $\tilde{\mathbf{w}} = \sum_{j \in S} \tilde{\mathbf{w}}_j$ is taken as input to generate the challenge c . In this approach, the adversary is prevented from manipulating the aggregated $\tilde{\mathbf{w}}$, as the coefficients β_i 's for the linear combination are determined by the random oracle H_β . However, the security proof of such a two-round construction is not straightforward, as directly invoking HVZK followed by programming is infeasible. Instead, existing approaches either rely on the non-standard AOM-MLWE assumption [EKT24] or involve embedding a trapdoor into $\{\mathbf{w}_{j,i}\}_{i \in [k]}$ [BKL⁺25].

Whether introducing a new assumption or embedding a trapdoor with the above approach, one must set the parameter k sufficiently large to ensure the security proof. From an efficiency standpoint, this results in high communication costs for both schemes, amounting to 275 KB [EKT24] and 618 KB [BKL⁺25], for $t = 1024$.

2.1 Our Solutions

Another idea for achieving a two-round protocol is to use a homomorphic trapdoor commitment scheme (HTDC), as in [DOTT21]. That is, in the first round, it sends a commitment to $\mathbf{w}_j$ and opens it in the second round upon revealing the response share $\mathbf{z}_j$. This inherently prevents the adversary from manipulating $\mathbf{w}$, as it can only tamper with the commitment, which provides no advantage, especially when the commitment key is message-dependent rather than fixed. In this way, when arguing the reduction from CMA security to NMA security, the reduction works in the trapdoor mode: it can firstly generate a commitment and later open it to a pre-determined value of $\mathbf{A}\mathbf{z}_j - \mathbf{c}\mathbf{b}_j + \Delta_j$. However, as noted in prior work [Che23], an HTDC-based signature must include the opening randomness $\mathbf{r}$, which significantly increases the signature size.

Our scheme TalonG adopts the concept of HTDC while addressing its limitations.

Weaken the Binding Requirement. A key observation is that the binding property in the standard commitment definition (which we refer to as standard binding) is not necessary; instead, a form of “*weak*” binding suffices for the commitment used in our construction. Concretely, we allow the adversary without trapdoor to find pairs of distinct $(\mathbf{w}^{(1)}, \mathbf{w}^{(2)})$ such that $\text{Com}(\mathbf{w}^{(1)}; \mathbf{r}^{(1)}) = \text{Com}(\mathbf{w}^{(2)}; \mathbf{r}^{(2)})$ with distinct randomness $(\mathbf{r}^{(1)}, \mathbf{r}^{(2)})$, but neither $\mathbf{w}^{(b)}$ can be expressed in the MLWE form $\mathbf{Az}^{(b)} - c^{(b)}\mathbf{b} + \Delta^{(b)}$, where $(\mathbf{A}, \mathbf{b})$ are predefined as public key components and $(c^{(b)}, \mathbf{z}^{(b)}, \Delta^{(b)})$ are short tuples of signatures ($b = 1, 2$). From an intuitive standpoint, weak binding is enough, since it prevents the adversary from committing to an arbitrary value and later opening it to a well-structured value of signature-verification form, which would lead to a forgery.

We take a closer look to the reduction of NMA security to the underlying MSIS problem. By applying the forking lemma [BN06], the reduction could find an MSIS solution via two valid forged signatures $(c^{(1)}, \mathbf{z}^{(1)}, \Delta^{(1)}, \mathbf{r}^{(1)})$ and $(c^{(2)}, \mathbf{z}^{(2)}, \Delta^{(2)}, \mathbf{r}^{(2)})$. Note that here the hash input to generate c is Com , so it holds that

$$\text{Com}(\mathbf{Az}^{(1)} - c^{(1)}\mathbf{b} + \Delta^{(1)}; \mathbf{r}^{(1)}) = \text{Com}(\mathbf{Az}^{(2)} - c^{(2)}\mathbf{b} + \Delta^{(2)}; \mathbf{r}^{(2)}). \quad (1)$$

In [DOTT21], the security proof relies on the standard binding: eq. (1) immediately gives another equation $\mathbf{Az}^{(1)} - c^{(1)}\mathbf{b} + \Delta^{(1)} = \mathbf{Az}^{(2)} - c^{(2)}\mathbf{b} + \Delta^{(2)}$, which leads to an MSIS solution.

However, standard binding is not necessary; weak binding suffices: any PPT adversary capable of producing values satisfying eq. (1) has directly broken the weak binding property, since the two committed values are both expressed in the MLWE form.

A straightforward way to achieve such *weak* binding is to use the form $\text{Com}(\mathbf{w}; \mathbf{r}) = \mathbf{w} + \mathbf{H}\mathbf{r}$ by simplifying [DOTT21]’s HTDC, where $\mathbf{H}$ is a uniformly random public matrix as the commitment key and $\mathbf{r}$ is randomness of small norm. The scheme satisfies weak binding: when a PPT adversary generates values satisfying eq. (1), it follows that

$$\mathbf{Az}^{(1)} - c^{(1)}\mathbf{b} + \Delta^{(1)} + \mathbf{H}\mathbf{r}^{(1)} = \mathbf{Az}^{(2)} - c^{(2)}\mathbf{b} + \Delta^{(2)} + \mathbf{H}\mathbf{r}^{(2)}.$$

As long as $\mathbf{H}$ is pre-set by the reductionist as the input to the MSIS problem (with a certain non-negligible probability), the above equation also leads to an MSIS solution $(\Delta^{(1)} - \Delta^{(2)}, \mathbf{z}^{(1)} - \mathbf{z}^{(2)}, c^{(1)} - c^{(2)}, \mathbf{r}^{(1)} - \mathbf{r}^{(2)})$ w.r.t. matrix $[\mathbf{A} \parallel -\mathbf{b} \parallel \mathbf{H}]$.

By removing the dependence on the standard binding, we achieve a more streamlined form of commitment, as illustrated above. More significantly, this opens the door to a more efficient trapdoor type that can further reduce the sizes of the commitment and the randomness.

Efficient NTRU-Type Trapdoor Semi-Commitment. Following the trapdoor type [MP12] used in [DOTT21], embedding the trapdoor into the matrix $\mathbf{H}$ requires $\mathbf{H}$ to be of large dimension, which in turn increases both the communication cost and the signature size. Therefore, we explore a new type of trapdoor.

In this paper, we employ a Falcon-style function $f_h(r_1, r_2) = r_1 + hr_2$ with the two short polynomials (r_1, r_2) as input, where Falcon [PFH⁺22] is NIST’s standard signature based on the NTRU problem. As a well-studied lattice assumption, NTRU [SS11, PS21] assumes that the distribution of $h = f^{-1}g \in \mathcal{R}_q$ is computationally indistinguishable from the uniform distribution over $\mathcal{R}_q$. Using the trapdoor $\mathbf{B}_{f,g}$ constructed with short f, g , we can get the preimage (r_1, r_2) of any function output. Based on such an NTRU-type trapdoor, we can construct a trapdoor *semi-commitment* scheme. We refer to this as a *semi-commitment* because it satisfies only weak binding rather than the standard binding property. On the other hand, it has much more compact commitment and randomness sizes than the scheme presented in [DOTT21].

- Commit: Under the commitment key h with $h \leftarrow \mathcal{R}_q$, for a message $\mu \in \mathcal{R}_q$ and randomness $\rho = (r_1, r_2) \in \mathcal{R}^2$ sampled from distribution $D_{\sigma_r}^2$, compute $\tilde{w} = \mu + f_h(r_1, r_2)$. Output $\text{com} = \tilde{w}$.
- Open: Given $\text{com} = \tilde{w}$, message μ , and randomness $\rho = (r_1, r_2)$, determine that com is valid if and only if $\tilde{w} = \mu + f_h(r_1, r_2)$ and (r_1, r_2) is short.
- Trapdoor-based Equivocability: When h is generated along with the trapdoor $\text{td} = \mathbf{B}_{f,g}$, given $(\text{td}, \text{com} = \tilde{w}, \mu)$, there exists a Falcon preimage sampling algorithm, which can sample $(r_1, r_2) \leftarrow D_{\sigma_r}^2$ s.t. $f_h(r_1, r_2) = \tilde{w} - \mu$.

Specifically, the semi-commitment is hiding since $f_h(r_1, r_2)$ is statistically close to uniform over $\mathcal{R}_q$. It is also weakly binding: constructing an equation of the form $az - bc + \Delta + f_h(r_1, r_2) = az' - bc' + \Delta' + f_h(r'_1, r'_2)$ is difficult under the RSIS assumption, where (a, b) will be defined as the public key and (c, z, Δ) as short components of signatures; see Section 5.3 for details. Its commitment com consists of a single polynomial, and the randomness involves only two short polynomials. We believe that this NTRU-type trapdoor commitment scheme can be directly leveraged to enhance the HTDC-based constructions in [DOTT21, GKS24].

Our Two-Round Construction. To ensure compatibility with the NTRU-type commitment scheme, our TalonG is based on the Ring LWE and Ring SIS assumptions, rather than the Module variants of LWE and SIS used in prior works, in which the committed message is exactly a polynomial⁴. The outline of TalonG proceeds as follows:

1. **Key Generation:** A trusted party samples $a \leftarrow \mathcal{R}_q$ uniformly and (s, e) from a bounded distribution over $\mathcal{R}_q^2$. It computes $b = as + e$ and uses Shamir’s secret sharing to distribute s and e into $\{s_j\}_{j \in [n]}$ and $\{e_j\}_{j \in [n]}$.

⁴ It can of course be extended to commit to multiple elements of the polynomial ring. However, from an application perspective, such an extension offers no practical benefit in terms of communicational efficiency.

The public key is $\text{vk} = (a, b)$, and each signer j receives the secret key share $\text{sk}_j = (s_j, e_j)$.

2. **Round 1:** Each signer j samples ephemeral randomness (y_j, e'_j) and commitment randomness $(r_{j,1}, r_{j,2})$ from a suitable distribution, and computes the share $w_j = ay_j + e'_j$ and the corresponding commitment $\tilde{w}_j = w_j + r_{j,1} + hr_{j,2}$, where $h = H_1(\text{msg} \parallel \text{vk})$ ⁵ and H_1 is a random oracle. It broadcasts $\tilde{w}_j$.
3. **Round 2:** Each signer j computes the challenge $c = H_2(\sum_{i \in S} \tilde{w}_i \parallel \text{msg} \parallel \text{vk})$ and the response share

$$(\tilde{z}_j, \tilde{\Delta}_j) = (y_j + \lambda_j s_j c + m_j, e'_j + \lambda_j e_j c + m'_j),$$

where m_j, m'_j are zero-sum masking terms, and H_2 is a random oracle. It broadcasts $(\tilde{z}_j, \tilde{\Delta}_j, r_{j,1}, r_{j,2})$.

4. **Combine:** The combiner aggregates the shares

$$(z, \Delta, r_1, r_2) = (\sum_{i \in S} \tilde{z}_i, \sum_{i \in S} \tilde{\Delta}_i, \sum_{i \in S} r_{i,1}, \sum_{i \in S} r_{i,2}),$$

and outputs the signature (c, z, Δ, r_1, r_2) .

5. **Verify:** The verifier computes $\tilde{w} = az - bc + \Delta + r_1 + hr_2$, and accepts the signature iff $c = H_2(\tilde{w} \parallel \text{msg} \parallel \text{vk})$ and (z, Δ, r_1, r_2) is short.

Reduce the size via Modulus Switching. During the computation of $\tilde{w} = az - bc + \Delta + r_1 + hr_2$, we observe that the term $\Delta + r_1$ is small. We can remove $\Delta + r_1$ and only output the difference between the high-order bits of $\tilde{w}$ and $az - bc + hr_2$. This can help us further significantly reduce the signature size. To realize it, we compress $\tilde{w}$ and $az - bc + \Delta$ via modulus switching. Specifically, the input to H_2 becomes $(\lfloor \tilde{w} \rfloor_p \parallel \text{msg} \parallel \text{vk})$, where $\lfloor \tilde{w} \rfloor_p$ denotes the result of applying modulus switching to $\tilde{w}$ by dropping its p least significant bits. Note that both modulus switching and modulus rounding can achieve the effect of dropping p bits, but our use of the former allows for additional bits of compression in the public key and signature sizes compared to the latter (see Lemma 2). We reconstruct an approximation via $\tilde{\Delta} = \lfloor \tilde{w} \rfloor_p - \lfloor az - bc + hr_2 \rfloor_p$, so that the signature no longer needs to include Δ and r_1 explicitly—only z, r_2 and $\tilde{\Delta}$ are required.

Proof Intuition. In the simulation of the first round, the challenger picks $\tilde{w}_j$ uniformly over $\mathcal{R}_q$, which is statistically close to the true distribution of $f_h(r_{j,1}, r_{j,2})$. In the second round, the responses $(\tilde{z}_j, \tilde{\Delta}_j)$ are sampled uniformly for all honest parties except the last one. In particular, the last honest party's $(w_{\text{last}}, \tilde{z}_{\text{last}}, \tilde{\Delta}_{\text{last}})$ is simulated differently: the challenger uses the RLWE's hints $y_{\text{sid}} = y_j + sc$ and $e_{\text{sid}} = e'_j + ec$ to generate $(w_{\text{last}}, \tilde{z}_{\text{last}}, \tilde{\Delta}_{\text{last}})$, where the distribution of y_j, e'_j is identical to that in the real protocol and (s, e) is the full signing key. After determining the committed message w_{last} , the challenger uses

⁵ For different messages msg , we use distinct commitment keys, which serves to prevent the concurrent attack [DEF⁺19].

the Falcon preimage sampling algorithm with trapdoor td to find the valid randomness.

It should be noted that h is a message-dependent commitment key. This serves not only to prevent concurrent attacks but is also required for the reduction. For the reduction to succeed, it must hold that: in the adversary's signing queries, h (as the output of the random oracle) is embedded into the trapdoor, while in the adversary's forgery, the corresponding h serves as the input to the RSIS problem, enabling a reduction to this problem without assuming the standard binding. We will analyze this success probability in detail in the formal security proof.

3 Preliminaries

3.1 Notations

Let $\lambda \in \mathbb{N}$ be the security parameter. Given $n \in \mathbb{N}$, we denote $[n]$ the set $\{1, 2, \dots, n\}$. Given a distribution $\mathcal{D}$, we write $x \leftarrow \mathcal{D}$ to denote that x is sampled from $\mathcal{D}$. Given a set R , we write $x \leftarrow R$ to denote that x is sampled from the uniform distribution over R . For a modulus $q \in \mathbb{N}$ and a power-of-two polynomial degree $N \in \mathbb{N}$, we define $\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z} = \{x + q\mathbb{Z} \mid x \in \mathbb{Z}\}$, $\mathcal{R} := \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$. Scalars, vectors and matrices over $\mathbb{Z}_q$ or $\mathcal{R}_q$ are typeset in italic (i.e., x), bold lowercase (i.e., $\mathbf{x}$) and bold uppercase (i.e., $\mathbf{X}$) fonts, respectively. We denote a vector with all components equal to 1 using the bold lowercase $\mathbf{1}$. For any vector $\mathbf{x} \in \mathbb{R}^m$, the l_1 -norm, l_2 -norm and l_∞ -norm are denoted by $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|$ and $\|\mathbf{x}\|_\infty$. When considering vectors $\mathbf{x} \in \mathcal{R}^m$, these norms are interpreted through the coefficient embedding of $\mathbf{x}$ into a standard Euclidean space. For $\mathbf{a} \in \mathbb{Z}_q^m$, where $\mathbb{Z}_q^m = \{\mathbf{x} + q\mathbb{Z}^m \mid \mathbf{x} \in \mathbb{Z}^m\}$ denotes the m -dimensional integer ring modulo q , we define the norm of a vector class $\|\mathbf{a}\|$ as $\min_{\mathbf{x} \in \mathbf{a} + q\mathbb{Z}^m} \|\mathbf{x}\|$. This definition extends naturally to the quotient ring $\mathcal{R}_q$ through its coefficient embedding. We now show the triangle inequality:

Lemma 1. *For any $q, m \in \mathbb{N}$, and $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_q^m$, we have $|\|\mathbf{x}\| - \|\mathbf{y}\|| \leq \|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.*

Proof. We represent $\mathbf{x}$ and $\mathbf{y}$ as equivalence classes $\mathbf{x} = \mathbf{x}_0 + q\mathbb{Z}^m$ and $\mathbf{y} = \mathbf{y}_0 + q\mathbb{Z}^m$ with $\mathbf{x}_0, \mathbf{y}_0 \in \mathbb{Z}^m$ satisfying $\|\mathbf{x}_0\|_\infty, \|\mathbf{y}_0\|_\infty \leq q/2$. By construction, $\|\mathbf{x}_0\| = \|\mathbf{x}\|$, $\|\mathbf{y}_0\| = \|\mathbf{y}\|$. For the upper bound:

$$\|\mathbf{x} + \mathbf{y}\| = \min_{\mathbf{z} \in \mathbf{x} + \mathbf{y}} \|\mathbf{z}\| \leq \|\mathbf{x}_0 + \mathbf{y}_0\| \leq \|\mathbf{x}_0\| + \|\mathbf{y}_0\| = \|\mathbf{x}\| + \|\mathbf{y}\|.$$

For the lower bound, observe that $\|\mathbf{x}\| - \|\mathbf{y}\| = \|(\mathbf{x} + \mathbf{y}) + (-\mathbf{y})\| - \|\mathbf{y}\| \leq \|\mathbf{x} + \mathbf{y}\|$.

This result extends naturally to the space $\mathbb{Q}^m/q\mathbb{Z}^m = \{\mathbf{x} + q\mathbb{Z}^m \mid \mathbf{x} \in \mathbb{Q}^m\}$.

3.2 Modulus Switching

In contrast to the modulus rounding techniques employed by del Pino et al. [dPKM⁺24,BKL⁺25], our construction instead leverages modulus switching to achieve a more compact bound of signatures. Let $q, q' \in \mathbb{N}$ and $q > q'$. We define the modulus switching function:

$$\lfloor \cdot \rfloor_{q'/q} : \mathbb{Z}_q \rightarrow \mathbb{Z}_{q'} \text{ s.t. } \lfloor x + q\mathbb{Z} \rfloor_{q'/q} := \lfloor q'x/q \rfloor + q'\mathbb{Z},$$

where $\lfloor \cdot \rfloor$ denotes the rounding operator. The function extends naturally to the vector space $\mathbb{Z}_q^m$, $\mathcal{R}_q$ and $\mathbb{Q}^m/q\mathbb{Z}^m$. When $q' = \lfloor q/2^p \rfloor$ for some $p \in \mathbb{N}$, we adopt the short notation $\lfloor \cdot \rfloor_p := \lfloor \cdot \rfloor_{q'/q}$.

We now present several lemmas that facilitate bound calculations in our constructions.

Lemma 2. *Let q, p be positive integers such that $q > 2^p$ and set $q' = \lfloor q/2^p \rfloor$.*

Then, for any $\mathbf{x} \in \mathbb{Z}_q^m$, we have $\left\| \mathbf{x} - \frac{q}{q'} \cdot \lfloor \mathbf{x} \rfloor_p \right\| \leq \frac{q}{q'} \cdot \left\| \frac{\mathbf{1}}{2} \right\|$.

Furthermore, for any $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_q^m$, the following inequality holds:

$$\left\| \mathbf{x} - \mathbf{y} - \frac{q}{q'} \cdot \left\| \lfloor \mathbf{x} \rfloor_p - \lfloor \mathbf{y} \rfloor_p \right\| \right\| \leq \frac{q}{q'} \cdot \|\mathbf{1}\|.$$

Proof. We first establish a preliminary claim, for any $\mathbf{z} \in \mathbb{Z}_{q'}^m$, $\frac{q}{q'} \cdot \|\mathbf{z}\| = \left\| \frac{q}{q'} \cdot \mathbf{z} \right\|$.

Proof of claim: decompose $\mathbf{z}$ into its equivalence classes representative $\mathbf{z} = \mathbf{z}_0 + q'\mathbb{Z}^m$ with $\mathbf{z}_0 \in \mathbb{Z}^m$ satisfying $\|\mathbf{z}_0\|_\infty \leq q'/2$. Since $\left\| \frac{q}{q'} \cdot \mathbf{z}_0 \right\|_\infty \leq q/2$,

$$\frac{q}{q'} \cdot \|\mathbf{z}\| = \frac{q}{q'} \cdot \|\mathbf{z}_0\| = \left\| \frac{q}{q'} \cdot \mathbf{z}_0 \right\| = \left\| \frac{q}{q'} \cdot \mathbf{z} \right\|.$$

For the first inequality, decompose $\mathbf{x}$ into its equivalence classes representative $\mathbf{x} = \mathbf{x}_0 + q\mathbb{Z}^m$ with $\mathbf{x}_0 \in \mathbb{Z}^m$ satisfying $\|\mathbf{x}_0\|_\infty \leq q/2$. Then:

$$\begin{aligned} \left\| \mathbf{x} - \frac{q}{q'} \cdot \lfloor \mathbf{x} \rfloor_p \right\| &= \left\| \mathbf{x}_0 - \frac{q}{q'} \cdot \left\lfloor \frac{q'\mathbf{x}_0}{q} \right\rfloor + q\mathbb{Z}^m \right\| \\ &= \left\| \frac{q}{q'} \cdot \left(\frac{q'\mathbf{x}_0}{q} - \left\lfloor \frac{q'\mathbf{x}_0}{q} \right\rfloor \right) \right\| \leq \frac{q}{q'} \cdot \left\| \frac{\mathbf{1}}{2} \right\|. \end{aligned}$$

For the second inequality, apply the triangle inequality (Lemma 1):

$$\begin{aligned} \left\| \mathbf{x} - \mathbf{y} - \frac{q}{q'} \cdot \left\| \lfloor \mathbf{x} \rfloor_p - \lfloor \mathbf{y} \rfloor_p \right\| \right\| &= \left\| \mathbf{x} - \mathbf{y} - \left\| \frac{q}{q'} \cdot \lfloor \mathbf{x} \rfloor_p - \frac{q}{q'} \cdot \lfloor \mathbf{y} \rfloor_p \right\| \right\| \\ &\leq \left\| \mathbf{x} - \frac{q}{q'} \cdot \lfloor \mathbf{x} \rfloor_p + \frac{q}{q'} \cdot \lfloor \mathbf{y} \rfloor_p - \mathbf{y} \right\| \\ &\leq 2 \cdot \frac{q}{q'} \cdot \left\| \frac{\mathbf{1}}{2} \right\| = \frac{q}{q'} \cdot \|\mathbf{1}\|. \end{aligned}$$

Remark 1. Compared with the modulus rounding techniques employed by del Pino et al. [dPKM⁺24,BKL⁺25], our modulus switching reduces the rounding error term from 2^p to $q/(2q') \approx 2^{p-1}$. This improvement allows for additional bits to be dropped from the signature construction.

3.3 Lattice and Discrete Gaussian

A lattice is a discrete set $\Lambda(\mathbf{b}_1, \dots, \mathbf{b}_n) = \{\sum_{i=1}^n x_i \mathbf{b}_i \mid x_i \in \mathbb{Z}\}$. For $\mathbf{x} \in \mathbb{R}^m$ or $\mathbf{x} \in \mathcal{R}_q^m$, and $\sigma > 0$, define the Gaussian function with parameter σ as $\rho_\sigma(\mathbf{x}) := \exp(-\pi \|\mathbf{x}\|^2 / \sigma^2)$. The discrete Gaussian distribution with parameter σ over lattice Λ is defined as $D_{\Lambda, \sigma}(\mathbf{x}) := \frac{\rho_\sigma(\mathbf{x})}{\sum_{\mathbf{x}' \in \Lambda} \rho_\sigma(\mathbf{x}')}$. When $\Lambda = \mathcal{R}_q$ (resp. $\mathcal{R}_q^m$), we simply denote D_σ (resp. D_σ^m).

Lemma 3 (Adapted from [Lyu12], Lemma 4.4). *Let $\sigma > 0, k > 1/\sqrt{2\pi}$, it holds that*

1. $\Pr[|z| > k\sigma : z \leftarrow D_{\mathbb{Z}, \sigma}] \leq 2 \exp(-\pi k^2)$.
2. $\Pr[\|z\| > k\sigma\sqrt{N} : z \leftarrow D_\sigma] \leq (\sqrt{2\pi}k)^N \cdot \exp(\frac{N}{2}(1 - 2\pi k^2))$.

In particular, setting $k = \frac{\exp(1/5)}{\sqrt{2\pi}}$, it holds that $\Pr[\|z\| > \frac{\exp(1/5)}{\sqrt{2\pi}} \sigma\sqrt{N} : z \leftarrow D_\sigma] \leq 2^{-N/16}$, which will be frequently used in subsequent constructions.

The ϵ -smoothing parameter of a lattice Λ , denoted by η_ϵ , is defined as $\eta_\epsilon(\Lambda) := \inf \{s > 0 : \rho_{1/s}(\Lambda^*) \leq 1 + \epsilon\}$.

3.4 Hardness Assumptions

We first recall a set of computational assumptions.

Definition 1 (RSIS). *Let $\mathcal{R}, k, N, q, \beta$ be parametrised by security parameter λ . The **RSIS** $_{\mathcal{R}, q, k, \beta}$ assumption states that for any PPT algorithm $\mathcal{A}$, the following probability is negligible in λ :*

$$\text{Adv}_{\mathcal{R}, q, k, \beta}^{\text{rsis}}(\lambda) = \left| \Pr \left[\mathbf{y} \leftarrow \mathcal{A}(\mathbf{a}) : \begin{array}{l} \mathbf{a} \leftarrow \mathcal{R}_q^{k-1}, \mathbf{y} \in \mathcal{R}_q^k, \\ \|\mathbf{y}\| \leq \beta, [\mathbf{1} \ \mathbf{a}^T] \mathbf{y} = 0 \end{array} \right] \right|$$

Definition 2 (NTRU). *Let $\mathcal{R}, N, q, \sigma$ be parametrised by security parameter λ . The **NTRU** $_{\mathcal{R}, q, \sigma}$ assumption states that for any PPT algorithm $\mathcal{A}$, the following probability is negligible in λ :*

$$\text{Adv}_{\mathcal{R}, q, \sigma}^{\text{ntru}}(\lambda) = \left| \Pr [1 \leftarrow \mathcal{A}(h) : (f, g) \leftarrow D_\sigma^2, h = f^{-1}g] - \Pr [1 \leftarrow \mathcal{A}(h) : h \leftarrow \mathcal{R}] \right|$$

Definition 3 (RLWE). *Let $\mathcal{R}, k, N, q, \sigma$ be parametrised by security parameter λ . The **RLWE** $_{\mathcal{R}, q, k, \sigma}$ assumption states that for any PPT algorithm $\mathcal{A}$, the following probability is negligible in λ :*

$$\text{Adv}_{\mathcal{R}, q, k, \sigma}^{\text{rlwe}}(\lambda) = \left| \Pr \left[1 \leftarrow \mathcal{A}(\mathbf{a}, \mathbf{b}) : \begin{array}{l} \mathbf{a} \leftarrow \mathcal{R}_q^k, s \leftarrow D_\sigma, \\ \mathbf{e} \leftarrow D_\sigma^k, \mathbf{b} = \mathbf{a}s + \mathbf{e} \end{array} \right] - \Pr \left[1 \leftarrow \mathcal{A}(\mathbf{a}, \mathbf{b}) : \begin{array}{l} \mathbf{a} \leftarrow \mathcal{R}_q^k, s \leftarrow D_\sigma, \\ \mathbf{e} \leftarrow D_\sigma^k, \mathbf{b} \leftarrow \mathcal{R}_q^k \end{array} \right] \right|$$

We recall the hint-LWE problem that allows an adversary to obtain noisy leakages of the LWE secrets. This problem was introduced in [KLSS23].

Definition 4 (Hint-RLWE). Let $\mathcal{R}, k, N, q, \sigma_1, \sigma_2, \tau, Q$ be parametrized by the security parameter λ . Let C denote a subset of $\mathcal{R}_q$ such that any $c \in \mathcal{R}$ satisfies $\|c\|_1 = \tau$. The **Hint-RLWE** $_{\mathcal{R}, q, k, \sigma_1, \sigma_2, \tau, Q}$ assumption states that for any PPT algorithm $\mathcal{A}$, the following probability is negligible in λ :

$$\begin{aligned} & \text{Adv}_{\mathcal{R}, q, k, \sigma_1, \sigma_2, \tau, Q}^{\text{hint-rlwe}}(\lambda) \\ &= \left| \Pr \left[1 \leftarrow \mathcal{A}(\mathbf{a}, \mathbf{b}, \{c_i, \mathbf{ht}_i\}_{i \in [Q]}) : \begin{array}{l} \mathbf{a} \leftarrow \mathcal{R}_q^k, s \leftarrow D_{\sigma_1}, \mathbf{e} \leftarrow D_{\sigma_1}^k, \\ y_i \leftarrow D_{\sigma_2}, \mathbf{e}'_i \leftarrow D_{\sigma_2}^k \\ \mathbf{b} = \mathbf{a}s + \mathbf{e}, \mathbf{ht}_i = \begin{pmatrix} y_i \\ \mathbf{e}'_i \end{pmatrix} + c_i \begin{pmatrix} s \\ \mathbf{e} \end{pmatrix} \end{array} \right] \right. \\ & \quad \left. - \Pr \left[1 \leftarrow \mathcal{A}(\mathbf{a}, \mathbf{b}, \{c_i, \mathbf{ht}_i\}_{i \in [Q]}) : \begin{array}{l} \mathbf{a} \leftarrow \mathcal{R}_q^k, s \leftarrow D_{\sigma_1}, \mathbf{e} \leftarrow D_{\sigma_1}^k, \\ y_i \leftarrow D_{\sigma_2}, \mathbf{e}'_i \leftarrow D_{\sigma_2}^k \\ \mathbf{b} \leftarrow \mathcal{R}_q^k, \mathbf{ht}_i = \begin{pmatrix} y_i \\ \mathbf{e}'_i \end{pmatrix} + c_i \begin{pmatrix} s \\ \mathbf{e} \end{pmatrix} \end{array} \right] \right| \end{aligned}$$

Under appropriate parameters, the hardness of Hint-RLWE can be reduced to that of RLWE [KLSS23, dPKM⁺24]. The concrete requirement is as follows.

Theorem 1 (Hardness of Hint-RLWE, [KLSS23]). Let k, q, τ, Q be positive integers, and σ_1, σ_2 positive real numbers. Define $\sigma > 0$ by the relation: $\frac{1}{\sigma^2} = \frac{2}{\sigma_1^2} + \frac{2Q\tau^2}{\sigma_2^2}$. If $\sigma > \sqrt{2} \cdot \eta_\epsilon(\mathbb{Z}^N)$, then there exists an efficient reduction from **RLWE** $_{\mathcal{R}, q, k, \sigma}$ to **Hint-RLWE** $_{\mathcal{R}, q, k, \sigma_1, \sigma_2, \tau, Q}$ that reduces the advantage by at most $(k+1) \cdot 2\epsilon$.

Remark 2 (Relaxed parameter relation). By [dPKM⁺24, Lemma B.2], the relation between σ and (σ_1, σ_2) can be relaxed to $\frac{1}{\sigma^2} = \frac{2}{\sigma_1^2} + \frac{2Q\tau \cdot (1+o(1))}{\sigma_2^2}$. This allows for choosing smaller values of σ_1, σ_2 to reduce the size of the signature while maintaining the same security level of **RLWE** $_{\mathcal{R}, q, k, \sigma}$.

Next, we introduce a lemma that will be useful in the security proof.

Lemma 4 (Adapted from [LPR13], Corollary 7.5). For any $a \in \mathcal{R}_q$, let $\mathcal{D}_{q, \sigma}^{\text{rlwe}}(a)$ be the distribution defined as $\{as + e | (s, e) \leftarrow D_\sigma\}$. Let $\mathcal{U}(\mathcal{R}_q)$ be uniform random distribution over $\mathcal{R}_q$. Assume that $\sigma > 2\sqrt{N} \cdot q^{1/2+1/N}$, then with probability $1 - 2^{-N}$ over the choice of a , we have $\Delta(\mathcal{D}_{q, \sigma}^{\text{rlwe}}(a), \mathcal{U}(\mathcal{R}_q)) \leq 2^{-N}$.

3.5 Pseudorandom Functions

We formally define the pseudorandom functions PRF as the description in [dPKM⁺24, Definition A.6].

Definition 5 (Pseudorandom Function (adapted from [dPKM⁺24])).

Let x, y be positive integers denoting input and output lengths. A deterministic PPT algorithm $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^x \rightarrow \{0, 1\}^y$ is a pseudorandom function if for any efficient adversary $\mathcal{A}$, the following advantage of $\mathcal{A}$ is $\text{negl}(\lambda)$:

$$\text{Adv}^{\text{prf}}(\lambda) = |\Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}_{\text{prf}}(\cdot)}(1^\lambda)] - \Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}_{\text{rf}}(\cdot)}(1^\lambda)]|$$

where the oracles are defined as:

$\mathcal{O}_{\text{prf}}(i, x)$	$\mathcal{O}_{\text{rf}}(i, x)$
1 : if $L_0[i] = \perp$	1 : if $L_1[i, x] = \perp$
2 : $\text{seed} \leftarrow \{0, 1\}^\lambda$	2 : $z \leftarrow \{0, 1\}^y$
3 : $L_0[i] := \text{seed}$	3 : $L_1[i, x] := z$
4 : $\text{seed} := L_0[i]$	4 : $z := L_1[i, x]$
5 : return $\text{PRF}(\text{seed}, x)$	5 : return z

3.6 NTRU-type Trapdoor

We revisit the foundational work of Falcon [PFH⁺22], which uses an NTRU-type trapdoor to construct a hash-and-sign lattice-based signature scheme. Specifically, our scheme adapts the Falcon function $f_h(r_1, r_2) = r_1 + h \cdot r_2 \bmod q$ to mask a ring element, since the distribution of the output $f_h(r_1, r_2)$ is indistinguishable from the uniform distribution over $\mathcal{R}_q$. At the same time, we exploit the following polynomial-time algorithms: trapdoor generation Falcon.GenTrap and preimage sampling Falcon.SamplePre , which are crucial for subsequent security analysis.

Theorem 2. Let q be a prime modulus, ϵ a negligible function in the security parameter λ , $\sigma_r > 1.17\sqrt{q} \cdot \sqrt{\log(4N(1 + 1/\epsilon))/\pi}$ a Gaussian parameter for the input (r_1, r_2) , and $\sigma_{td} > 1.17\sqrt{q/(2N)}$ a Gaussian parameter for trapdoor sampling. There exist PPT algorithms $(\text{Falcon.GenTrap}, \text{Falcon.SamplePre})$ satisfying:

- $\text{Falcon.GenTrap}(1^\lambda)$: On input security parameter 1^λ , output commitment key $\text{Fck} = h \in \mathcal{R}_q$ and trapdoor $\text{Ftd} = \mathbf{B}_{f,g} \in \mathcal{R}_q^{2 \times 2}$ structured as: $\mathbf{B}_{f,g} = \begin{pmatrix} f & F \\ g & G \end{pmatrix}$ with properties:
 1. $f^{-1}g = h$, where $(f, g) \leftarrow D_{\sigma_{td}}^2$.
 2. $fG - gF = q$
 3. $\|\mathbf{B}_{f,g}\|_{\text{GS}} \leq 1.17\sqrt{q}$

- $\text{Falcon.SamplePre}(h, \mathbf{B}_{f,g}, w)$: Given $\text{Fck} = h$, $\text{Ftd} = \mathbf{B}_{f,g}$, and target $w \in \mathcal{R}_q$, output $(r_1, r_2) \in \mathcal{R}_q^2$ satisfying $r_1 + h \cdot r_2 = w$. The algorithm ensures that the following two distributions are computationally indistinguishable:

$$\left\{ (h, w, r_1, r_2) : \begin{array}{l} h \leftarrow \mathcal{R}_q, (r_1, r_2) \leftarrow D_{\sigma_r}^2, \\ w = r_1 + h \cdot r_2 \end{array} \right\}$$

$$\left\{ (h, w, r_1, r_2) : \begin{array}{l} (h, \mathbf{B}_{f,g}) \leftarrow \text{Falcon.GenTrap}, w \leftarrow \mathcal{R}_q, \\ (r_1, r_2) \leftarrow \text{Falcon.SamplePre}(h, \mathbf{B}_{f,g}, w) \end{array} \right\}.$$

The distinguishing advantage $\text{Adv}_{\text{ind}}^{\text{falcon}}(\lambda)$ is bounded by:

$$\text{Adv}_{\text{ind}}^{\text{falcon}}(\lambda) \leq \text{Adv}_{\mathcal{R}, q, \sigma_{td}}^{\text{ntru}}(\lambda) + (N + 2) \cdot \epsilon(\lambda).$$

The formal proof of Theorem 2 is provided in Appendix A.

3.7 Threshold Signatures

We show the syntax of the R -round threshold signature following the definition. $n \in \mathbb{N}$ is the number of all signers. $t \in \mathbb{N}$ is the reconstruction threshold s.t. $t \leq n$. We use act to denote an active signer set such that $\text{act} \subseteq [n]$ with size t . Each signer $j \in [n]$ keeps a state st_j to retain the information specified for the short session. Formally, an R -round (t, n) -threshold signature scheme is a tuple of PPT algorithms $(\text{KGen}, \{\text{ShareSign}_i\}_{i \in [R]}, \text{Combine}, \text{Vf})$.

- $\text{KGen}(1^\lambda, t, n) \rightarrow (\text{vk}, \{\text{sk}_j\}_{j \in [n]})$: A trusted party takes as inputs security parameters 1^λ , reconstruction threshold t and total number of signers n . Then it generates a verification key vk and secret key shares $\{\text{sk}_j\}_{j \in [n]}$ for each signer. Each signer's state is initialized as $\text{st}_j := \emptyset$.
- $\text{ShareSign}_i(\text{sk}_j, \text{st}_j, \text{act}, \text{msg}, \text{pm}_{i-1}) \rightarrow (\text{st}_j, \text{pm}_i[j])$: In the i -th ($i \in [R]$) round signing protocol, the user j uses secret key sk_j , current state st_j , an active signer set act , a message msg and protocol message in previous round $\text{pm}_{i-1} = \{\text{pm}_{i-1}[j]\}_{j \in [\text{act}]}$ to produce a new protocol message $\text{pm}_i[j]$ and updated state st_j . Especially, the protocol messages pm_0 are empty.
- $\text{Combine}(\text{vk}, \text{msg}, \{\text{pm}_i\}_{i \in [R]}) \rightarrow \text{sig}$: The combiner aggregates all last round protocol messages of all signers in act into a final signature sig .
- $\text{Vf}(\text{vk}, \text{msg}, \text{sig})$: The verifier takes as inputs the public key vk , a message msg and a signature sig . Then it outputs 1 if sig is valid, 0 otherwise.

Definition 6 (Correctness of Threshold Signature). An R -round threshold signature scheme is correct if for all $1 \leq t \leq n$, for all active signer sets $\text{act} \subset [n]$ with $|\text{act}| \geq t$ and for all messages $\text{msg} \in \{0, 1\}^*$, the following probability satisfies:

Game_A^{tuf}(1^λ)
1 : ST := ∅
2 : C ← A(1 ^λ)
3 : if C ⊄ [n] or C ≥ t, return 0
4 : H := [n] \ C
5 : (vk, {sk _j } _{j∈[n]}) ← KGen(1 ^λ)
6 : (msg*, sig*) ← A ^{{OS_i}_{i∈[R]}, H} (vk, {sk _j } _{j∈C})
7 : if msg* ∈ ST or Vf(vk, msg*, sig*) = 0
8 : return 0
9 : else return 1
OS _i (j, act, msg, pm _{i-1})
1 : if j ∉ H ∩ act, return ⊥
2 : ST := ST ∪ {msg}
3 : (st _j , pm _i [j]) ← ShareSign _i (sk _j , st _j , act, msg, pm _{i-1})
4 : return pm _i [j]

Fig. 1: Unforgeability Game

$$\Pr \left[\begin{array}{l} (vk, \{sk_j\}_{j \in [n]}) \leftarrow \text{KGen}(1^\lambda, t, n) \\ \forall i \in [R], j \in \text{act}, (st_j, pm_i[j]) \\ \leftarrow \text{ShareSign}_i(sk_j, st_j, \text{act}, \text{msg}, pm_{i-1}) \\ \text{sig} \leftarrow \text{Combine}(vk, \text{msg}, \{pm_i\}_{i \in [R]}) \end{array} \right] = 1 - \text{negl}(\lambda)$$

We formalize the security of threshold signature schemes under selective unforgeability adversarial models. A PPT adversary $\mathcal{A}$ can corrupt at most $t - 1$ signers, obtaining their secret keys $\{sk_j\}_{j \in \mathcal{C}}$ where $\mathcal{C}$ denotes the corrupted set with $|\mathcal{C}| \leq t - 1$. For the remaining honest set $\mathcal{H} := [n] \setminus \mathcal{C}$, $\mathcal{A}$ can interactively query partial signing oracles to obtain protocol messages $\{pm_i\}$ on arbitrary messages of its choice. Unforgeability is formally defined as follows:

Definition 7 (Unforgeability of Threshold Signature). *An R -round threshold signature scheme is unforgeable if for any PPT adversaries $\mathcal{A}$ (with access to the random oracle H), the following probability is negligible in λ :*

$$\text{Adv}^{\text{tuf}}(\lambda) := \Pr[\text{Game}_{\mathcal{A}}^{\text{tuf}}(1^\lambda) = 1],$$

where the game $\text{Game}_{\mathcal{A}}^{\text{tuf}}$ is defined in [Figure 1](#).

4 Weakly Binding Commitment (Semi-Commitment)

We construct a homomorphic trapdoor commitment scheme with a weak binding property (i.e., semi-commitment) based on the aforementioned NTRU-type trapdoor in Section 3.6. Following [DOTT21]’s syntax, it has the following algorithms:

- $\text{CGen}(1^\lambda) \rightarrow h$: The key generation algorithm outputs $h \leftarrow \mathcal{R}_q$ as the commitment key.
- $\text{Com}_h(\mu; \rho) \rightarrow \text{com}$: For a message $\mu \in \mathcal{R}_q$ and randomness $\rho = (r_1, r_2) \leftarrow D_{\sigma_r}^2$, the committing algorithm outputs $\text{com} = \mu + f_h(r_1, r_2) \in \mathcal{R}_q$.
- $\text{Open}_h(\text{com}, \mu, \rho) \rightarrow \{0, 1\}$: Given $\text{com}, \mu, \rho = (r_1, r_2)$, the opening algorithm outputs 1 if and only if $\text{com} = \mu + f_h(r_1, r_2)$ and the vector (r_1, r_2) satisfies the required norm bounds. Otherwise, it outputs 0.
- $\text{TCGen}(1^\lambda) \rightarrow (h, \mathbf{B}_{f,g})$: The trapdoor key generation algorithm runs Falcon.
- $\text{GenTrap}(1^\lambda) \rightarrow (h, \mathbf{B}_{f,g})$.
- $\text{Eqv}_h(\mathbf{B}_{f,g}, \text{com}, \mu) \rightarrow (r_1, r_2)$: If h is generated together with the trapdoor $\mathbf{B}_{f,g}$, the trapdoor-based equivocation algorithm outputs $(r_1, r_2) \leftarrow \text{Falcon.SamplePre}(h, \mathbf{B}_{f,g}, \text{com} - \mu)$.

Given the properties of the Falcon function $f_h(\cdot)$, it is evident that the commitment is homomorphic and satisfies the hiding property. The equivocability can be obtained directly from the property of Falcon.SamplePre in [Theorem 2](#). However, it does not have the standard binding. One can trivially find collisions by setting $\mu_1 = 0$ and $\mu_2 = [1 \ h](\rho_1 - \rho_2)$, which satisfies $\text{Com}_h(\mu_1; \rho_1) = \text{Com}_h(\mu_2; \rho_2)$.

Specifically, the scheme satisfies the *weak binding* property. We begin by defining this property.

Definition 8 (Weak Binding). *A commitment scheme $(\text{CGen}, \text{Com}, \text{Open})$ is weakly binding if for any PPT adversary $\mathcal{A}$, a norm bound $\beta \in \mathbb{R}^+$,*

$$\Pr \left[\begin{array}{l} \mu_i = az_i - bc_i + \Delta_i \\ \text{Com}_h(\mu_1; \rho_1) = \text{Com}_h(\mu_2; \rho_2) : \\ \|(z_i, c_i, \Delta_i, \rho_i)\| \leq \beta \end{array} : \begin{array}{l} (a, b) \leftarrow \mathcal{R}_q^2, h \leftarrow \text{CGen}(1^\lambda) \\ (z_i, c_i, \Delta_i, \rho_i) \leftarrow \mathcal{A}(a, b, h) \\ i \in \{1, 2\} \end{array} \right] = \text{negl}(\lambda)$$

Our commitment scheme is weakly binding. Suppose an adversary outputs $(z_1, c_1, \Delta_1, \rho_1)$ and $(z_2, c_2, \Delta_2, \rho_2)$ such that $\|(z_i, c_i, \Delta_i, \rho_i)\| \leq \beta$ and

$$az_1 - bc_1 + \Delta_1 + [1 \ h]\rho_1 = az_2 - bc_2 + \Delta_2 + [1 \ h]\rho_2.$$

Rearranging yields $az' - bc' + hr' + \Delta' = 0$ with $\|(z', -c', r', \Delta')\| \leq 2\beta$. This directly gives an RSIS solution for the public vector $[1 \ a \ b \ h]$.

It is important to note that the constraints imposed on μ_i within Definition 8 are highly tailored to the specific lattice-based signature structure. As such, it is a primitive designed specifically for our threshold signature scheme. However, by applying different algebraic constraints on μ_i , this definition could be adapted for broader use cases. Establishing a more generalized definition and exploring these wider applications remain subjects for our future work.

5 Our Threshold Signature Scheme

5.1 Our Construction

This section presents our construction of a t -out-of- n lattice-based threshold signature scheme in Figure 2. We present the scheme's parameters in Table 2. Additionally, we use two hash functions $H_1 : \{0, 1\}^* \rightarrow \mathcal{R}_q$, $H_2 : \{0, 1\}^* \rightarrow C$ both modeled as random oracles and a pseudorandom function $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \mathcal{R}_q$. Our threshold signature scheme consists of five polynomial-time algorithms: key generation KGen , two-round partial signing $\text{ShareSign}_1, \text{ShareSign}_2$, signature combination Combine , and verification Vf .

Note that our construction and security proof implicitly rely on the semi-commitment scheme introduced in Section 4. For a more intuitive presentation, we integrate the specific structure of the semi-commitment scheme directly into our threshold signature design, rather than treating it as a black-box instantiation.

Table 2: Parameters for our threshold signature.

Parameter	Explanation
λ	Security parameter
q	Ring modulus
t, n	Threshold value and the number of all parties
N	Degree of polynomial
$\mathcal{R}_q$	Polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$
p_1	Number of bits dropping performed on verification key b
p_2	Number of bit dropping performed on aggregated commitment w
(q_1, q_2)	$q_1 = \lfloor q/2^{p_1} \rfloor$, $q_2 = \lfloor q/2^{p_2} \rfloor$, ring modulus after bit dropping
$\lfloor x \rfloor_{p_1}$	For $x \in \mathcal{R}_q$, $\lfloor x \rfloor_{p_1} = \lfloor q_1 x / q \rfloor \in \mathcal{R}_{q_1}$
$\lfloor x \rfloor_{p_2}$	For $x \in \mathcal{R}_q$, $\lfloor x \rfloor_{p_2} = \lfloor q_2 x / q \rfloor \in \mathcal{R}_{q_2}$
σ_1	Gaussian parameter of secret key (s, e)
σ_2	Gaussian parameter of ephemeral randomness (y_i, e'_i)
σ_r	Gaussian parameter of commitment randomness r_i
D_σ	Discrete Gaussian distribution with parameter σ in $\mathcal{R}_q$
(C, τ)	ternary polynomial challenge set C with l_1 -norm τ
$(\beta_z, \beta_r, \beta_\Delta)$	l_2 -norm bound on signature z, r, Δ
(Q_S, Q_H)	Max number of signing queries and hash queries

Key Generation. As with most lattice-based threshold signature schemes [dPKM⁺24, EKT24, BKL⁺25, dPN25], our key generation phase requires a trusted dealer to distribute the secret shares. It proceeds as follows:

1. *Public Key Derivation:* A trusted party samples a uniform ring element $a \leftarrow \mathcal{R}_q$ and a short secret pair $(s, e) \leftarrow D_{\sigma_1}^2$ from a discrete Gaussian distribution. The public key component is then computed as $b = \lfloor as + e \rfloor_{p_1} \in \mathcal{R}_{q_1}$.

KGen ($1^\lambda, t, n$)	ShareSign ₁ (sk_j, st_j, msg)
1 : $a \leftarrow \mathcal{R}_q, (s, e) \leftarrow D_{\sigma_1}^2, \hat{b} := as + e$ 2 : $b := \lfloor \hat{b} \rfloor_{p_1}$ 3 : $\{s_i\}_{i \in [n]} \leftarrow \text{SS.Share}(s)$ 4 : for $i, j = 1$ to n do 5 : $seed_{i,j} \leftarrow \{0, 1\}^\lambda$ 6 : for $i = 1$ to n do 7 : $sk_i := (s_i, (seed_{i,j}, seed_{j,i})_{j \in [n]})$ 8 : $vk := (a, b)$ 9 : return $(vk, (sk_i)_{i \in [n]})$	1 : $(y_j, e'_j) \leftarrow D_{\sigma_2}^2, (r_{j,1}, r_{j,2}) \leftarrow D_{\sigma_r}^2$ 2 : $h := H_1(msg \ vk)$ 3 : $w_j := ay_j + e'_j + r_{j,1} + hr_{j,2}$ 4 : $st_j := st_j \cup \{(msg, w_j, y_j, r_{j,2})\}$ 5 : return $pm_1[j] = w_j$
Combine (vk, msg, pm_1, pm_2)	ShareSign ₂ ($sk_j, st_j, act, msg, pm_1$)
1 : $z := \sum_{i \in \text{act}} z_i, r := \sum_{i \in \text{act}} r_{i,2}, w := \lfloor \sum_{i \in \text{act}} w_i \rfloor_{p_2}$ 2 : $h := H_1(msg \ vk), c := H_2(w \ msg \ vk)$ 3 : $\Delta := w - \lfloor az - \frac{q}{q_1}bc + hr \rfloor_{p_2}$ 4 : return $sig = (z, c, r, \Delta)$	1 : if $ act < t$ or $act \not\subseteq [n]$, return $\perp$ 2 : for $i \in act$ do 3 : $w_i := pm_1[i]$ 4 : if $\nexists (msg, w_j, *) \in st_j$, return $\perp$ 5 : Fetch $(msg, w_j, y_j, r_{j,2})$ from st_j 6 : $w := \lfloor \sum_{i \in act} w_i \rfloor_{p_2}$ 7 : $c := H_2(w \ msg \ vk)$ 8 : for $i \in act$ do 9 : $p_{i,j} := \text{PRF}(seed_{i,j}, act \ msg \ pm_1)$ 10 : $p_{j,i} := \text{PRF}(seed_{j,i}, act \ msg \ pm_1)$ 11 : $m_j = \sum_{i \in act} (p_{i,j} - p_{j,i})$ 12 : $z_j := y_j + \lambda_{act,j} s_j c + m_j$ 13 : $st_j := st_j \setminus \{(msg, w_j, y_j, r_{j,2})\}$ 14 : return $pm_2[j] = (z_j, r_{j,2})$
Vf ($vk, msg, sig = (z, c, r, \Delta)$)	
1 : $h := H_1(msg \ vk)$ 2 : $w := \lfloor az - \frac{q}{q_1}bc + hr \rfloor_{p_2} + \Delta$ 3 : if $c = H_2(w \ msg \ vk)$ & $\ z\ \leq \beta_z$ 4 : & $\ r\ \leq \beta_r$ & $\ \Delta\ \leq \beta_\Delta$ 5 : return 1 6 : else return 0	

Fig. 2: Two-Round Lattice-based Threshold Signature Scheme

2. *Secret Sharing*: In order to implement a t -out-of- n threshold signature scheme, the secret s is divided into n shares $\{s_i\}_{i \in [n]}$ via Shamir's secret sharing scheme, which are then distributed to n signers. Any subset $\text{act} \subset [n]$ with $|\text{act}| \geq t$ can reconstruct s via Lagrange interpolation $s = \sum_{i \in \text{act}} \lambda_{\text{act},i} s_i \mod q$, where $\lambda_{\text{act},i} = \prod_{k \in \text{act} \setminus \{i\}} k/(k-i) \mod q$.
3. *Pairwise Seed Distribution*: For each pair $(i, j) \in [n]^2$, the trusted party generates independent seeds $\text{seed}_{i,j} \leftarrow \{0, 1\}^\lambda$ to enable pseudorandom function (PRF) during partial signing.
4. *Key Distribution*: The verification key $\text{vk} = (a, b)$ is published, while each signer i receives a secret share $\text{sk}_i = (s_i, \{\text{seed}_{i,j}, \text{seed}_{j,i}\}_{j \in [n]})$.

Two-Round Signing. Our signing protocol follows a lattice-based Fiat-Shamir framework. To realize two-round communication in a distributed manner, we use *weakly binding commitment* defined in Section 4 to mask each party's "commit" component $ay_j + e'_j$, where the corresponding NTRU trapdoor helps us in completing the security proof, instead of relying on an additional round to perform a hash-and-open paradigm as [dPKM⁺24].

In detail, the first round signing algorithm proceeds as follows:

1. *Commitment Generation*: Each signer j first generates a message-specific key $h = H_1(\text{msg} \parallel \text{vk}) \in \mathcal{R}_q$, then generates its share $w_j = ay_j + e'_j + r_{j,1} + hr_{j,2} \mod q$. The ephemeral randomness (y_j, e'_j) are sampled from $D_{\sigma_2}^2$, which are utilized to mask the private key (s, e) . Another randomness $(r_{j,1}, r_{j,2})$ are sampled from $D_{\sigma_r}^2$, which are utilized to mask the fresh RLWE sample $ay_j + e'_j \mod q$.
2. *State Management*: The tuple $(\text{msg}, w_j, y_j, r_{j,2})$ is stored in st_j . The first round protocol message $\text{pm}_1[j]$ is defined as w_j .

The second round signing algorithm proceeds as follows:

1. *Consistency Check*: Each signer j takes as input its secret key sk_j , current state st_j , active signer set act , message msg and the first round protocol message pm_1 . It confirms $|\text{act}| \geq t$ and the existence of the session state $(\text{msg}, \text{pm}_1[j], *)$, ensuring participation in the first round but not in the second round.
2. *Challenge Derivation*: Each signer j computes the rounded aggregated commitment $w = \lfloor \sum_{i \in \text{act}} w_i \rfloor_{p_2}$ and the challenge $c = H_2(w \parallel \text{msg} \parallel \text{vk})$.
3. *Response Generation*: Each signer j constructs zero-sum masking term $\sum_{i \in \text{act}} (p_{i,j} - p_{j,i})$ through pairwise PRF seeds with the session-specific input $\text{act} \parallel \text{msg} \parallel \text{pm}_1$ ⁶. The signer further computes the response share $z_j := y_j + \lambda_{\text{act},j} s_j c + \sum_{i \in \text{act}} (p_{i,j} - p_{j,i}) \mod q$.

⁶ Note that the honest parties will not use the same PRF input to obtain identical outputs across two executions, since the commitment share w_j in pm_1 has high min-entropy.

4. *State Management*: The tuple $(\text{msg}, w_j, y_j, r_{j,2})$ is removed from st_j . The second round protocol message $\text{pm}_2[j]$ is defined as $(z_j, r_{j,2})$.

Remark 3 (No need for view authentication). In [dPKM⁺24], an additional standard signature scheme is needed for view authentication. The purpose of view authentication is to ensure that all honest parties have a consistent view for the first-round message pm_1 . In this paper, the usage of PRFs can guarantee that only if all parties share the same view on pm_1 , the sum of the m_j values is zero. Otherwise, the sum is a pseudorandom value, and the responses remain random from the view of the adversary, and thus can be simulated. Consequently, a view-authentication step is no longer required.

Signature Combination. The combiner aggregates partial signatures from subset act :

1. Commitment aggregation $w = \lfloor \sum_{i \in \text{act}} w_i \rfloor_{p_2}$.
2. Response aggregation $z = \sum_{i \in \text{act}} z_i$.
3. Randomness aggregation $r = \sum_{i \in \text{act}} r_{i,2}$.
4. Hint $\Delta = w - \lfloor az - \frac{a}{q_1}bc + hr \rfloor_{p_2}$ allows the verifier to check the validity of the signature. Output final signature $\text{sig} = (z, c, r, \Delta)$.

Verification. To verify signature (z, c, r, Δ) on the message msg , the verifier first reconstructs $h := H_1(\text{msg} \parallel \text{vk})$ and $w := \Delta + \lfloor az - \frac{a}{q_1}bc + hr \rfloor_{p_2}$. It then checks whether $c = H_2(w \parallel \text{msg} \parallel \text{vk})$ and (z, c, r, Δ) satisfy the boundary conditions.

5.2 Correctness Analysis

This section establishes bounds $(\beta_z, \beta_r, \beta_\Delta)$ within the verification procedure. Let $\hat{b} := as + e$ denote the original public key, and define the rounding error term as $\hat{e} := \hat{b} - \frac{a}{q_1}b$, which satisfies $\|\hat{e}\|_\infty \leq \frac{q}{2q_1}$.

Under honest users' behaviors, we derive the following properties:

1. The aggregated response satisfies

$$z = \sum_{i \in \text{act}} (y_i + \lambda_{\text{act},i} s_i c + \sum_{j \in \text{act}} (p_{j,i} - p_{i,j})) = \sum_{i \in \text{act}} y_i + sc.$$

Given $y_i \leftarrow D_{\sigma_2}$, the convolution property of discrete Gaussian distributions implies $\sum_{i \in \text{act}} y_i \leftarrow D_{\sqrt{t}\sigma_2}$ with $|\text{act}| = t$. For $s \leftarrow D_{\sigma_1}$ and c containing τ entries of ± 1 , the product sc approximates $sc \leftarrow D_{\sqrt{\tau}\sigma_1}$. Combining these results, we set $\beta_z = \alpha\sqrt{N}(\sqrt{t}\sigma_2 + \sqrt{\tau}\sigma_1)$, where $\alpha = \frac{\exp(1/5)}{\sqrt{2\pi}}$ as specified in Lemma 3.

2. The aggregated randomness satisfies $r = \sum_{i \in \text{act}} r_{i,2}$. For $r_{i,2} \leftarrow D_{\sigma_r}$, the convolution property gives $\sum_{i \in \text{act}} r_{i,2} \leftarrow D_{\sqrt{t}\sigma_r}$. We thus set $\beta_r = \alpha\sqrt{tN}\sigma_r$.

3. The aggregated commitment satisfies

$$\begin{aligned}
w &= \lfloor \sum_{i \in \text{act}} (ay_i + e'_i + r_{i,1} + hr_{i,2}) \rfloor_{p_2} \\
&= \lfloor az - asc + \sum_{i \in \text{act}} (e'_i + r_{i,1}) + hr \rfloor_{p_2} \\
&= \lfloor az - (\hat{e} + \frac{q}{q_1}b - e)c + hr + \sum_{i \in \text{act}} (e'_i + r_{i,1}) \rfloor_{p_2} \\
&= \lfloor az - \frac{q}{q_1}bc + hr + \tilde{e} \rfloor_{p_2}
\end{aligned}$$

where $\tilde{e} := \sum_{i \in \text{act}} (e'_i + r_{i,1}) - (\hat{e} - e)c$ represents residual error terms. We bound $\tilde{e}$ by:

- For $\sum_{i \in \text{act}} (e'_i + r_{i,1}) + ec$: we set the bound $\alpha\sqrt{N}(\sqrt{t}(\sigma_2 + \sigma_r) + \sqrt{\tau}\sigma_1)$.
 - For $-\hat{e}c$: we set the bound $\|\hat{e}c\| \leq \sqrt{N}\|\hat{e}\|_\infty\|c\|_1 \leq \frac{q\tau}{2q_1}\sqrt{N}$.
- Combining these components gives $\tilde{e}$'s bound:

$$\beta_{\tilde{e}} = \alpha\sqrt{N}(\sqrt{t}(\sigma_2 + \sigma_r) + \sqrt{\tau}\sigma_1) + \frac{q\tau}{2q_1}\sqrt{N}.$$

4. The difference hint satisfies

$$\Delta = \lfloor az - \frac{q}{q_1}bc + hr + \tilde{e} \rfloor_{p_2} - \lfloor az - \frac{q}{q_1}bc + hr \rfloor_{p_2}.$$

Applying [Lemma 2](#) yields $\|\Delta\| \leq \frac{q_2}{q}\|\tilde{e}\| + \|\mathbf{1}\|$, leading to the composite bound:

$$\beta_{\Delta} = \frac{q_2}{q}\beta_{\tilde{e}} + \sqrt{N} = \frac{\alpha q_2 \sqrt{N}}{q}(\sqrt{t}(\sigma_2 + \sigma_r) + \sqrt{\tau}\sigma_1) + (\frac{q_2\tau}{2q_1} + 1)\sqrt{N}.$$

Theorem 3 (Correctness). *Assume $N = 16\lambda$ and let the bounds $(\beta_z, \beta_r, \beta_{\Delta})$ be defined as above. The threshold signature scheme in [Figure 2](#) satisfies correctness with overwhelming probability:*

$$\Pr[(\|z\| \leq \beta_z) \wedge (\|r\| \leq \beta_r) \wedge (\|\Delta\| \leq \beta_{\Delta})] \geq 1 - 3 \cdot 2^{-\lambda}.$$

Proof. We establish the correctness bound through a union argument over the three failure events. Recall the bounds are configured such that:

$$\Pr[\|z\| > \beta_z] \leq 2^{-\lambda}, \Pr[\|r\| > \beta_r] \leq 2^{-\lambda}.$$

$$\Pr[\|\Delta\| > \beta_{\Delta}] \leq \Pr[\|\tilde{e}\| > \beta_{\tilde{e}}] \leq 2^{-\lambda}.$$

Applying the union bound to the complementary events:

$$\begin{aligned}
&1 - \Pr[(\|z\| \leq \beta_z) \wedge (\|r\| \leq \beta_r) \wedge (\|\Delta\| \leq \beta_{\Delta})] \\
&= \Pr[(\|z\| > \beta_z) \vee (\|r\| > \beta_r) \vee (\|\Delta\| > \beta_{\Delta})] \\
&\leq \Pr[\|z\| > \beta_z] + \Pr[\|r\| > \beta_r] + \Pr[\|\tilde{e}\| > \beta_{\tilde{e}}] \leq 3 \cdot 2^{-\lambda}.
\end{aligned}$$

5.3 Security Analysis

In this section, we establish the security of our threshold signature scheme (formally defined in Figure 2) through a sequence of hybrid games. Denote $\mathcal{H}_{\text{act}} := \mathcal{H} \cap \text{act}$ as the set of honest active signers and $\mathcal{C}_{\text{act}} := \mathcal{C} \cap \text{act}$ as the set of corrupted active signers.

Theorem 4 (Unforgeability). *Let $\epsilon = 2^{-\lambda}$, $\sigma_2 > 2\sqrt{N} \cdot q^{1/2+1/N}$, $\sigma_r > 1.17\sqrt{q} \cdot \sqrt{\log(4N(1+1/\epsilon))}/\pi$, $\sigma_{td} > 1.17\sqrt{q/(2N)}$. Let σ satisfy $\frac{1}{\sigma^2} = \frac{2}{\sigma_1^2} + \frac{2Q_H\tau}{\sigma_2^2}$ with $\sigma > \sqrt{2} \cdot \eta_\epsilon(\mathbb{Z}^N)$. Let $N, \beta_z, \beta_r, \beta_\Delta$ defined as Theorem 3. Let Q_S and Q_H denote the upper bounds on the number of signing oracles OS_1, OS_2 queries and random oracles H_1, H_2 queries, respectively. The threshold signature scheme described in Figure 2 is unforgeable under the following assumptions:*

- The pseudorandomness of PRF.
- $\text{NTRU}_{\mathcal{R},q,\sigma_{td}}$ assumption.
- $\text{RLWE}_{\mathcal{R},q,k,\sigma}$ assumption with $k = 1$, $\frac{1}{\sigma^2} = \frac{2}{\sigma_1^2} + \frac{2Q_H\tau}{\sigma_2^2}$.
- $\text{RSIS}_{\mathcal{R},q,k,\beta}$ assumption with $k = 4$, $\beta = \sqrt{4\beta_z^2 + 4\beta_r^2 + 4\tau + \beta_\delta^2}$, $\beta_\delta = \frac{q}{q_2}(\sqrt{N} + 2\beta_\Delta) + \frac{q\tau}{q_1}\sqrt{N}$.

Concretely, for any PPT adversary $\mathcal{A}$,

$$\begin{aligned} \text{Adv}^{\text{tuf}}(\lambda) \leq & \text{Adv}^{\text{prf}}(\lambda) + \frac{1}{2^{N-1}} + e(Q_S + 1) \cdot \left[Q_S \cdot (\text{Adv}_{\mathcal{R},q,\sigma_{td}}^{\text{ntru}}(\lambda) + (N + 2) \cdot \epsilon) \right. \\ & \left. + \text{Adv}_{\mathcal{R},q,1,\sigma}^{\text{rlwe}}(\lambda) + (2k + 2) \cdot \epsilon + \frac{Q_H^2}{|\mathcal{C}|} + Q_H^{3/2} \cdot \sqrt{\text{Adv}_{\mathcal{R},q,4,\beta}^{\text{rsis}}(\lambda)} \right]. \end{aligned}$$

Proof Overview. The detailed security proof is provided in Appendix B. Before showing the full proof, we show a proof sketch via a brief game-hopping argument.

Hybrid₀: It is the original security game defined in Definition 7.

Hybrid₁: It changes the way in answering the oracle OS_2 . It uses random values to replace the PRF outputs of honest parties. This change is indistinguishable due to the pseudo-randomness of PRF.

Hybrid₂: Instead of calculating $m_j = \sum_{i \in \text{act}} (p_{i,j} - p_{j,i})$, it samples the mask $m_j \leftarrow \mathcal{R}_q$ for all honest parties $j \in \mathcal{H}_{\text{act}}$, except for the last honest party, whose m_j is set consistently as $m_j := -\sum_{k \in \mathcal{C}_{\text{act}}} m_k - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} m_k$. This is identical to Hybrid₁ since m_j 's still act as zero's random shares.

Hybrid₃: It changes the way in answering H_1 queries and the condition of returning 1. When answering the oracle $\text{H}_1(\text{msg} \parallel \text{vk})$, it invokes Falcon.GenTrap with probability ϕ and records the trapdoor corresponding to $\text{msg} \parallel \text{vk}$. Define the passing conditions: (i) whenever $\text{H}_1(\cdot)$ is invoked indirectly through the signing oracle OS_1 , a corresponding trapdoor is always recorded; (ii) no trapdoor is generated for a direct query $\text{H}_1(\text{msg}^* \parallel \text{vk})$, where msg^* is the message of the adversary $\mathcal{A}$'s forgery. By setting $\phi = Q_S/(Q_S + 1)$, the probability of passing is at least $1/e(Q_S + 1)$. So the probability that Hybrid₃ outputs 1 is no less than $1/e(Q_S + 1)$ times of Hybrid₂ outputs 1.

Hybrid₄: It changes the way in answering OS_1 queries. Instead of generating $w_j = ay_j + e'_j + r_{j,1} + hr_{j,2}$ honestly, it samples $w_j \leftarrow \mathcal{R}_q$ randomly and generates $(r_{j,1}, r_{j,2})$ via Falcon's preimage sampling algorithm `Falcon.SamplePre` with trapdoor. Due to the property of the NTRU-type trapdoor ([Theorem 2](#)), the adversary cannot notice it.

Hybrid₅: In answering OS_2 queries, it generates the response z_j without using honest parties' secret key shares. More precisely, it samples $z_j \leftarrow \mathcal{R}_q$ at random for all honest parties $j \in \mathcal{H}_{\text{act}}$, except for the last honest party, whose z_{last} is set

$$z_{\text{last}} = \sum_{k \in \mathcal{H}_{\text{act}}} y_k + (s - \sum_{k \in \mathcal{C}_{\text{act}}} \lambda_{\text{act},k} s_k) c - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{\text{last}\}} z_k.$$

It is identical to Hybrid₄: each honest party's z_j is random and their sum satisfy

$$\sum_{k \in \mathcal{H}_{\text{act}}} z_k = \sum_{k \in \mathcal{H}_{\text{act}}} y_k + (s - \sum_{k \in \mathcal{C}_{\text{act}}} \lambda_{\text{act},k} s_k) c - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}).$$

Hybrid₆: For the last honest party, it sets

$$z_{\text{last}} = y_{\text{sid}} + \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{\text{last}\}} (y_k - z_k) - \sum_{k \in \mathcal{C}_{\text{act}}} \lambda_{\text{act},k} s_k c - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}).$$

This is merely a rearrangement, due to $y_{\text{sid}} = y_{\text{last}} + sc$. Then, the corresponding $\hat{w}_{\text{last}} = w_{\text{last}} - (ay_{\text{last}} + e'_{\text{last}})$ can be generated by the following equation $\hat{w}_{\text{last}} = w_{\text{last}} - ay_{\text{sid}} - e_{\text{sid}} + \hat{b}c$ with $e_{\text{sid}} = e'_{\text{last}} + ec$ and $\hat{b} = as + e$. Thus, it is identical to Hybrid₅.

Hybrid₇: The final hybrid modifies the way of generating secret key shares for corrupted parties. It picks $\hat{b}$ uniformly random and computes $\{s_i\}_i$ as shares of 0 instead of s . From Hybrid₆, the challenger did not use secret key (s, e) or honest parties' secret key shares s_j to simulate the signing oracles. For corrupted parties, the shares of 0 distribute identically to those of s . By using the hint to compute the shares w_j and z_j , one can reduce the difference of Hybrid₆ and Hybrid₇ to the advantage of the hint-RLWE problem.

To analyze the adversary's success probability in Hybrid₇, we apply the forking lemma to transform the adversary's two forgeries $(z^*, c^*, r^*, \Delta^*)$ and (z', c', r', Δ') for the same message msg^* into a weak binding collision to $[a|\hat{b}|h^*]$, with $h^* := H_1(\text{msg}^* || \text{vk})$.

6 Performance

Parameter Settings. We implement our TalonG scheme under the following parameter settings.

- The degree of polynomial $N = 16\lambda$, the negligible function $\epsilon = 2^{-\lambda}$.

Table 3: **Parameters for our construction.** The sizes $|\text{vk}|$ and $|\text{sig}|$ are provided in KB. The values for RLWE/RSIS/NTRU indicate the security strength of the corresponding hard problem. All parameter sets satisfy $t = 1024, q \approx 2^{43}, Q_S = Q_H = 2^{60}$.

λ	N	τ	$(\sigma_1, \sigma_2, \sigma_r)$	(p_1, p_2)	$(\beta_z, \beta_r, \beta_\Delta)$	$ \text{vk} $	$ \text{sig} $	RLWE/RSIS/NTRU
128	2048	14	$(2^6, 2^{36}, 2^{24.2})$	$(35, 38)$	$(2^{45.5}, 2^{33.6}, 2^{8.0})$	2.0	17.7	158/128/573
256	4096	27	$(2^6, 2^{36}, 2^{24.6})$	$(36, 39)$	$(2^{45.9}, 2^{34.6}, 2^{8.2})$	3.5	35.4	372/265/512

- The Gaussian parameter for ephemeral randomness $\sigma_2 > 2\sqrt{N} \cdot q^{1/2+1/N}$.
- The Gaussian parameter for randomness $\sigma_r > 1.17\sqrt{q} \cdot \sqrt{\log(4N(1+1/\epsilon))}/\pi$.
- The Gaussian parameter for NTRU $\sigma_{td} > 1.17\sqrt{q/(2N)}$.
- The Gaussian parameter for RLWE $\frac{1}{\sigma^2} = \frac{2}{\sigma_1^2} + \frac{2Q_H\tau}{\sigma_2^2}$ with $\sigma > \sqrt{2} \cdot \eta_\epsilon(\mathbb{Z}^N)$.
- The verification bounds $(\beta_z, \beta_r, \beta_\Delta) = \left(\alpha\sqrt{N}(\sqrt{t}\sigma_2 + \sqrt{\tau}\sigma_1), \alpha\sqrt{tN}\sigma_r, \frac{\alpha q_2\sqrt{N}}{q}(\sqrt{t}(\sigma_2 + \sigma_r) + \sqrt{\tau}\sigma_1) + (\frac{q_2\tau}{2q_1} + 1)\sqrt{N} \right)$.
- The bound for RSIS $\beta_{SIS} := \sqrt{4\beta_z^2 + 4\beta_r^2 + 4\tau + \beta_\delta^2}$, where $\beta_\delta = \frac{q}{q_2}(\sqrt{N} + 2\beta_\Delta) + \frac{q\tau}{q_1}\sqrt{N}$.

According to the above theory, the concrete parameter sets employed in our scheme for 128-bit and 256-bit security are summarized in Table 3. The security strengths of RLWE, RSIS, and NTRU are estimated according to the lattice estimator⁷ of [APS15] with the cost model in [MAT22].

Table 1 compares our scheme with existing schemes in terms of communication overhead, and public-key and signature sizes at a security level of $\lambda = 128$ bits. Our scheme minimizes both communication and public-key size, while offering a signature size that remains comparable. Furthermore, Table 3 indicates that even at $\lambda = 256$ bits, our scheme continues to exhibit relatively small public-key and signature sizes, only 3.5 KB and 35.4 KB for $t = 1024$.

Implementation. We implemented TalonG under the 128-bit security parameter set on a desktop equipped with an Intel Core i7-13700 processor 2.10 GHz. All benchmarks for TalonG and Ringtail were conducted in a single-core setting to ensure a fair and consistent comparison, with each performance metric representing the average of 50 independent runs. Our implementation builds upon the open-source codebase of Ringtail, with modifications to incorporate our new semi-commitment structure and optimized signing protocol.

It should be noted that the performance data for TRaccoon are derived from the original publication [dPKM⁺24], whereas the results for TalonG and Ringtail are obtained from our own measurements under a unified testing environment. Therefore, the comparison with TRaccoon is not strictly controlled and should be interpreted as indicative. Nevertheless, our signing protocol is expected to

⁷ <https://github.com/malb/lattice-estimator> (up to commit 5ba00f5)

Table 4: **The computational costs (ms) for TRaccoon [dPKM⁺24], Ringtail [BKL⁺25], and our TalonG.** The DSign column shows the total running times of the signing protocol.

t	Scheme	KeyGen	DSign	Verify
4	TRaccoon	0.282	10.580	0.521
	Ringtail	4.180	62.354	0.868
	TalonG	2.250	1.040	0.815
16	TRaccoon	0.199	13.208	0.520
	Ringtail	7.151	87.257	0.884
	TalonG	6.840	2.967	0.925
64	TRaccoon	0.389	24.474	0.524
	Ringtail	22.009	182.277	0.755
	TalonG	23.568	9.454	0.929
256	TRaccoon	1.351	71.748	0.521
	Ringtail	81.733	605.782	0.715
	TalonG	87.927	35.916	0.756
1024	TRaccoon	5.472	252.244	0.527
	Ringtail	486.915	2487.886	0.716
	TalonG	491.589	140.713	0.876

be faster in theory, as it avoids the use of an additional signature algorithm for view authentication as required in TRaccoon (see Remark 3).

As summarized in Table 4, TalonG demonstrates significant performance improvements over Ringtail in the signing phase across all threshold values. Specifically, for $t = 1024$, our total signing time is 140.713 ms, which is 17.7 times faster than Ringtail’s 2487.886 ms. The improvement stems primarily from the reduction in the number of polynomials processed per interaction round, which substantially decreases the required polynomial multiplications—a dominant cost in lattice-based signing.

For key generation, our runtime is similar to Ringtail’s, but significantly slower than TRaccoon’s. This may be due to the differences in randomness generation or hardware, as analyzed by Ringtail. Since TRaccoon’s code is not open source, we cannot provide a precise analysis of the cause.

Verification times remain stable and efficient across all thresholds, consistently staying below 1 ms. This is because verification involves only a small, constant number of polynomial multiplications independent of t , with minor fluctuations arising from sampling noise.

References

- ADP24. Nabil Alkeilani Alkadri, Nico Döttling, and Sihang Pu. Practical lattice-based distributed signatures for a small number of signers. In *Applied Cryptography and Network Security - 22nd International Conference*,

- ACNS 2024, Abu Dhabi, United Arab Emirates, March 5-8, 2024, Proceedings, Part I*, volume 14583 of *Lecture Notes in Computer Science*, pages 376–402. Springer, 2024.
- APS15. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.
- ASY22. Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-Optimal Lattice-Based Threshold Signatures, Revisited. In Mikołaj Bojańczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 8:1–8:20, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- BCdP⁺25. Giacomo Borin, Sofia Celi, Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Threshold signatures reloaded: ML-DSA and enhanced raccoon with identifiable aborts. *Cryptology ePrint Archive*, Paper 2025/1166, 2025.
- BGG⁺18. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In *Advances in Cryptology – CRYPTO 2018*, pages 565–596, Cham, 2018. Springer International Publishing.
- BKL⁺25. Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. In *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*, pages 149–164. IEEE, 2025.
- BN06. Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS 2006, Alexandria, VA, USA, October 30 - November 3, 2006*, pages 390–399. ACM, 2006.
- BP25. Luís TAN Brandão and René Peralta. Nist first call for multi-party threshold schemes. <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8214C.2pd.pdf>, 2025.
- BTT22. Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. Musig-l: Lattice-based multi-signature with single-round online phase. In *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 276–305. Springer, 2022.
- CGG⁺20. Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In *CCS ’20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1769–1787. ACM, 2020.
- Che23. Yanbo Chen. sfdualms: Efficient lattice-based two-round multi-signature with trapdoor-free simulation. In *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, vol-

- ume 14085 of *Lecture Notes in Computer Science*, pages 716–747. Springer, 2023.
- CTZ24. Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part IV*, volume 15487 of *Lecture Notes in Computer Science*, pages 268–302. Springer, 2024.
- DEF⁺19. Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the security of two-round multi-signatures. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1084–1101. IEEE, 2019.
- Des90. Yvo Desmedt. Abuses in cryptography and how to fight them. In *Advances in Cryptology — CRYPTO’ 88*, pages 375–389, New York, NY, 1990. Springer New York.
- DF90. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In *Advances in Cryptology — CRYPTO’ 89 Proceedings*, pages 307–315, New York, NY, 1990. Springer New York.
- DKL⁺18. Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):238–268, 2018.
- DKL⁺23. Jack Doerner, Yashvanth Kondi, Eysa Lee, Abhi Shelat, and LaKiah Tyner. Threshold BBS+ signatures for distributed anonymous credential issuance. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 773–789. IEEE, 2023.
- DOTT21. Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. In *Public-Key Cryptography – PKC 2021*, pages 99–130, Cham, 2021. Springer International Publishing.
- dPKM⁺24. Rafael del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In *Advances in Cryptology – EUROCRYPT 2024*, pages 219–248, Cham, 2024. Springer Nature Switzerland.
- dPKN⁺25. Rafaël del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking raccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. In *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 423–456. Springer, 2025.
- dPKPR24. Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part I*, volume 14920 of *Lecture Notes in Computer Science*, pages 409–444. Springer, 2024.
- dPN25. Rafael del Pino and Guilhem Niot. Finally! a compact lattice-based threshold signature. In *Public-Key Cryptography – PKC 2025*, pages 169–199, Cham, 2025. Springer Nature Switzerland.

- DR24. Sourav Das and Ling Ren. Adaptively secure BLS threshold signatures from DDH and co-cdh. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 251–284. Springer, 2024.
- EKT24. Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In *Advances in Cryptology - CRYPTO 2024*, pages 387–424, Cham, 2024. Springer Nature Switzerland.
- ENP24. Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerge: Distributed key generation and robust threshold signature from lattices. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 425–458. Springer, 2024.
- GKPV10. Shafi Goldwasser, Yael Tauman Kalai, Chris Peikert, and Vinod Vaikuntanathan. Robustness of the learning with errors assumption. In *Innovations in Computer Science - ICS 2010, Tsinghua University, Beijing, China, January 5-7, 2010. Proceedings*, pages 230–240. Tsinghua University Press, 2010.
- GKS24. Kamil Doruk Gur, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In *Post-Quantum Cryptography*, pages 266–300, Cham, 2024. Springer Nature Switzerland.
- GPV08. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing, Victoria, British Columbia, Canada, May 17-20, 2008*, pages 197–206. ACM, 2008.
- KG20. Chelsea Komlo and Ian Goldberg. FROST: flexible round-optimized schnorr threshold signatures. In *Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2020.
- KLSS23. Duhyeon Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-mlwe. In *Advances in Cryptology - CRYPTO 2023*, pages 549–580, Cham, 2023. Springer Nature Switzerland.
- KRT24. Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 459–491. Springer, 2024.
- Lin24. Yehuda Lindell. Simple three-round multiparty schnorr signing with full simulatability. *IACR Commun. Cryptol.*, 1(1):25, 2024.
- LPR13. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-lwe cryptography. In *Advances in Cryptology - EUROCRYPT 2013*, pages 35–54, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- Lyu09. Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In *Advances in Cryptology - ASIACRYPT 2009*, pages 598–616, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

- Lyu12. Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Advances in Cryptology – EUROCRYPT 2012*, pages 738–755, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- MAT22. MATZOV. Report on the Security of LWE: Improved Dual Lattice Attack. 2022.
- MP12. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718. Springer, 2012.
- NRS21. Jonas Nick, Tim Ruffing, and Yannick Seurin. Musig2: Simple two-round schnorr multi-signatures. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 189–221. Springer, 2021.
- Oka92. Tatsuaki Okamoto. Provably secure and practical identification schemes and corresponding signature schemes. In *Advances in Cryptology - CRYPTO '92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings*, volume 740 of *Lecture Notes in Computer Science*, pages 31–53. Springer, 1992.
- PFH⁺22. Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon. Technical report, National Institute of Standards and Technology, 2022. Available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- PS21. Alice Pellet-Mary and Damien Stehlé. On the hardness of the NTRU problem. In *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part I*, volume 13090 of *Lecture Notes in Computer Science*, pages 3–35. Springer, 2021.
- Sho97. Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26(5):1484–1509, 1997.
- SS11. Damien Stehlé and Ron Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings*, volume 6632 of *Lecture Notes in Computer Science*, pages 27–47. Springer, 2011.
- THL⁺24. Guofeng Tang, Shuai Han, Li Lin, Changzheng Wei, and Ying Yan. Batch range proof: How to make threshold ECDSA more efficient. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 4256–4270. ACM, 2024.
- TPCZ23. Guofeng Tang, Bo Pang, Long Chen, and Zhenfeng Zhang. Efficient lattice-based threshold signatures with functional interchangeability. *IEEE Transactions on Information Forensics and Security*, 18:4173–4187, 2023.

- TX25. Guofeng Tang and Haiyang Xue. Robust threshold ECDSA with online-friendly design in three rounds. In *IEEE Symposium on Security and Privacy SP*, pages 203–221. IEEE, 2025.
- ZT25. Chenzhi Zhu and Stefano Tessaro. The algebraic one-more MISIS problem and applications to threshold signatures. In *CRYPTO*, volume 16000, pages 548–581. Springer, 2025.

A Proof of Theorem 2

In this section, we prove the indistinguishability of `Falcon.SamplePre` in Section 3.6 via a sequence of hybrid distributions. First, we set the primal distribution

$$\mathcal{D}_0 = \left\{ (h, w, r_1, r_2) : \begin{array}{l} h \leftarrow \mathcal{R}_q, (r_1, r_2) \leftarrow D_{\sigma_r}^2 \\ w = r_1 + h \cdot r_2 \end{array} \right\},$$

which is generated without using any trapdoor.

Define the first distribution:

$$\mathcal{D}_1 = \left\{ (h, w, r_1, r_2) : \begin{array}{l} (h, \mathbf{B}_{f,g}) \leftarrow \text{Falcon.GenTrap}(1^\lambda) \\ (r_1, r_2) \leftarrow D_{\sigma_r}^2, w = r_1 + h \cdot r_2 \end{array} \right\}.$$

The distribution $\mathcal{D}_1$ differs from $\mathcal{D}_0$ only in how h is sampled. Under NTRU assumption [SS11], h from $\mathcal{D}_0$ is computationally distinguishable from h in $\mathcal{D}_1$. For any PPT adversary $\mathcal{A}$,

$$|\Pr[\mathcal{A}(\mathcal{D}_0) = 1] - \Pr[\mathcal{A}(\mathcal{D}_1) = 1]| \leq \text{Adv}_{\mathcal{R},q,\sigma_{td}}^{\text{ntru}}(\lambda).$$

Define the second distribution:

$$\mathcal{D}_2 = \left\{ (h, w, r_1, r_2) : \begin{array}{l} (h, \mathbf{B}_{f,g}) \leftarrow \text{Falcon.GenTrap}(1^\lambda), w \leftarrow \mathcal{R}_q \\ (r_1, r_2) \leftarrow D_{\sigma_r}^2 | r_1 + h \cdot r_2 = w \end{array} \right\}.$$

Here, w is sampled uniformly and (r_1, r_2) is sampled from the conditional distribution. For fixed w , the conditional distributions of $\mathcal{D}_2$ and $\mathcal{D}_1$ are identical. Thus, the statistical distance between $\mathcal{D}_2$ and $\mathcal{D}_1$ arises only from the marginal distribution of w . By [GPV08, Corollary 2.8], when $\sigma_r > \eta_\epsilon(\Lambda_h^\perp)$, then

$$\{r_1 + h \cdot r_2 : r_1, r_2 \leftarrow D_r\}$$

is within statistical distance 2ϵ of the uniform distribution over $\mathcal{R}_q$, where

$$\Lambda_h^\perp = \{(s_1, s_2) \in \mathcal{R}_q^2 : s_1 + h \cdot s_2 = 0\}.$$

By [GPV08, lemma 3.1],

$$\eta_\epsilon(\Lambda_h^\perp) \leq \|\tilde{\mathbf{B}}_{f,g}\|_{\text{GS}} \cdot \sqrt{\log(4N(1 + 1/\epsilon))/\pi}.$$

Since $\|\tilde{\mathbf{B}}_{f,g}\|_{\text{GS}} \leq 1.17\sqrt{q}$. So $\sigma_r > \eta_\epsilon(\Lambda_h^\perp)$. Therefore,

$$\Delta(\mathcal{D}_1, \mathcal{D}_2) \leq 2\epsilon.$$

Define the third distribution:

$$\mathcal{D}_3 = \left\{ (h, w, r_1, r_2) : \begin{array}{l} (h, \mathbf{B}_{f,g}) \leftarrow \text{Falcon.GenTrap}(1^\lambda), w \leftarrow \mathcal{R}_q, \\ (r_1, r_2) \leftarrow \text{Falcon.SamplePre}(h, \mathbf{B}_{f,g}, w) \end{array} \right\}.$$

The distribution $\mathcal{D}_3$ differs from $\mathcal{D}_2$ only in how (r_1, r_2) is sampled. By the properties of Gaussian sampler in Falcon [GPV08, lemma 4.5], if $\sigma_r > \|\tilde{\mathbf{B}}_{f,g}\|_{\text{GS}} \cdot \eta_\epsilon(\mathcal{R}^2)$, then the statistical distance between the output of `Falcon.SamplePre` and the conditional distribution $D_{\sigma_r}^2 \mid r_1 + h_i r_2 = w$ is at most $N \cdot \epsilon$. Hence,

$$\Delta(\mathcal{D}_2, \mathcal{D}_3) \leq N\epsilon.$$

Combining the above, for any PPT adversary $\mathcal{A}$,

$$|\Pr[\mathcal{A}(\mathcal{D}_0) = 1] - \Pr[\mathcal{A}(\mathcal{D}_3) = 1]| \leq \text{Adv}_{\mathcal{R}, q, \sigma_{td}}^{\text{ntru}}(\lambda) + (N + 2) \cdot \epsilon.$$

B Proof of Theorem 4

In this section, we provide the concrete security proof of TalonG described in Figure 2. We prove this by a hybrid argument. The base game Hybrid_0 is described in Figure 3. We formalize the hybrid progression from Hybrid_1 to Hybrid_7 as described in Figure 4a to Figure 9. Denote $\mathcal{H}_{\text{act}} := \mathcal{H} \cap \text{act}$ as the set of honest active signers and $\mathcal{C}_{\text{act}} := \mathcal{C} \cap \text{act}$ as the set of corrupted active signers.

Hybrid₁. The challenger maintains a dedicated table `Signed[act||msg||pm1]` to record honest signers who have participated in the second round of signing protocol with the input $(\text{act}, \text{msg}, \text{pm}_1)$. It aborts if $j \in \text{Signed}[\text{act}||\text{msg}||\text{pm}_1]$ which means the oracle OS_2 has been queried with the input $(j, \text{act}, \text{msg}, \text{pm}_1)$. Otherwise, the sampling of PRF values $p_{i,j}, p_{j,i}$ is modified: when both parties i and j are honest, these values are sampled uniformly from $\mathcal{R}_q$. The table `Signed[act||msg||pm1]` prevents input-mapping conflicts, ensuring no two distinct outputs are assigned to the same input during the PRF simulation. It is depicted in Figure 4a.

Hybrid_1 is the same as Hybrid_0 , except the PRF simulation, and one bad event bad_1 : an adversary makes twice signature queries targeting the same valid⁸ input tuple $(j, \text{act}, \text{msg}, \text{pm}_1)$. The event bad_1 causes termination in Hybrid_1 . We demonstrate that the probability of this event is negligible.

Lemma 5. *The probability that the bad event bad_1 happens is $\Pr[\text{bad}_1] \leq 2^{-N+1}$.*

⁸ Valid input $(j, \text{act}, \text{msg}, \text{pm}_1)$ here implies that the oracle OS_2 in Hybrid_0 does not return $\perp$ upon receiving $(j, \text{act}, \text{msg}, \text{pm}_1)$.

Hybrid ₀	OS ₁ (j, msg)
<pre> 1: ST, HT1, HT2 := ∅ 2: Signed[·], TDT[·] := ⊥ 3: C ← A(1^λ) 4: if C ⊄ [n] or C ≥ t, return 0 5: H := [n] \ C 6: (vk, {sk_j}_{j∈[n]}) ← KGen(1^λ) 7: (msg*, sig*) ← A^{OS₁, OS₂, H₁, H₂}(vk, {sk_j}_{j∈C}) 8: if msg* ∈ ST or Vf(vk, msg*, sig*) = 0 9: return 0 10: else return 1 </pre>	<pre> 1: if j ∉ H, return ⊥ 2: ST := ST ∪ {msg} 3: (y_j, e'_j) ← D²_{σ₂}, (r_{j,1}, r_{j,2}) ← D²_{σ_r} 4: h := H₁(msg vk) 5: w_j := ay_j + e'_j + r_{j,1} + hr_{j,2} 6: st_j := st_j ∪ {(msg, w_j, y_j, r_{j,2})} 7: return pm₁[j] = w_j </pre>
H ₁ (str)	OS ₂ ($j, \text{act}, \text{msg}, \text{pm}_1$)
<pre> 1: if ∃ h s.t. (str, h) ∈ HT1 2: return h 3: else 4: h ← R_q 5: HT₁ := HT₁ ∪ {(str, h)} 6: return h </pre>	<pre> 1: if j ∉ H ∩ act, return ⊥ 2: if act < t or act ⊄ [n], return ⊥ 3: for i ∈ act do 4: w_i := pm₁[i] 5: if ∄(msg, w_j, *) ∈ st_j, return ⊥ 6: Fetch (msg, w_j, y_j, r_{j,2}) from st_j 7: w := ⌊ ∑_{i∈act} w_i ⌋_{p₂} 8: c := H₂(w msg vk) 9: for i ∈ act do 10: p_{i,j} := PRF(seed_{i,j}, act msg pm₁) 11: p_{j,i} := PRF(seed_{j,i}, act msg pm₁) 12: m_j := ∑_{i∈act} (p_{i,j} - p_{j,i}) 13: z_j := y_j + λ_{act,j} s_j c + m_j 14: st_j := st_j \ {(msg, w_j, y_j, r_{j,2})} 15: return pm₂[j] = (z_j, r_{j,2}) </pre>
H ₂ (str)	
<pre> 1: if ∃ c s.t. (str, c) ∈ HT2 2: return c 3: else 4: c ← C 5: HT₂ := HT₂ ∪ {(str, c)} 6: return c </pre>	

Fig. 3: Hybrid₀

Hybrid ₁	H ₁ (str)
// Identical to the Hybrid ₀	// Identical to the Hybrid ₀
OS ₁ (j, msg)	H ₂ (str)
// Identical to the Hybrid ₀	// Identical to the Hybrid ₀
OS ₂ (j, act, msg, pm ₁)	
<pre> 1: if j ∉ H ∩ act, return ⊥ 2: if act < t or act ∉ [n], return ⊥ 3: for i ∈ act do 4: w_i := pm₁[i] 5: if ∄(msg, w_j, *) ∈ st_j, return ⊥ 6: Fetch (msg, w_j, y_j, r_{j,2}) from st_j 7: if j ∈ Signed[act msg pm₁], return ⊥ 8: Signed[act msg pm₁] := Signed[act msg pm₁] ∪ {j} 9: w := ⌊ ∑_{i ∈ act} w_i ⌋_{p₂} 10: c := H₂(w msg vk) 11: for i ∈ C ∩ act do 12: p_{i,j} := PRF(seed_{i,j}, act msg pm₁) 13: p_{j,i} := PRF(seed_{j,i}, act msg pm₁) 14: for i ∈ H ∩ act do 15: if i ∉ Signed[act msg pm₁] 16: p_{i,j} ← R_q, Record[act msg pm₁ i j] := p_{i,j} 17: p_{j,i} ← R_q, Record[act msg pm₁ j i] := p_{j,i} 18: if i ∈ Signed[act msg pm₁] \ {j} 19: p_{i,j} := Record[act msg pm₁ i j] 20: p_{j,i} := Record[act msg pm₁ j i] 21: m_j := ∑_{i ∈ act} (p_{i,j} - p_{j,i}) 22: z_j := y_j + λ_{act,j}s_jc + m_j 23: st_j := st_j \ {(msg, w_j, y_j, r_{j,2})} 24: return pm₂[j] = (z_j, r_{j,2}) </pre>	

(a) Hybrid₁(b) Hybrid₂Fig. 4: Hybrid₁-Hybrid₂

Proof. Assume event [bad₁ occurs, meaning OS₁ outputs the same w_j for two distinct executions with input $(j, \text{act}, \text{msg}, \text{pm}_1)$. According to Lemma 4, with probability $1 - 2^{-N}$ over the choice of a , $H_\infty(ay_j + e'_j) \geq N$, so $H_\infty(w_j) \geq H_\infty(ay_j + e'_j) \geq N$ satisfies with overwhelming probability. The probability that OS₁ outputs w_j again is less than $2^{-N} + (1 - 2^{-N}) \cdot 2^{-H_\infty(w_j)} < 2^{-N+1}$.

Then, we use the following lemma to analyze the advantage difference between the two hybrids.

Lemma 6. *If there exists a PPT adversary $\mathcal{A}$ that distinguishes Hybrid₀ from Hybrid₁, then a PPT adversary $\mathcal{B}$ can be constructed to break the pseudorandom function PRF security. Especially,*

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_0}(\lambda) - \text{Adv}_{\mathcal{A}}^{\text{hybrid}_1}(\lambda) \leq \text{Adv}^{\text{prf}}(\lambda) + 2^{-N+1}.$$

Proof. We construct the reduction $\mathcal{B}$ as follows:

1. Computing $(vk, \{sk_i\}_{i \in [n]})$ identically to Hybrid_0 and provide $(vk, \{sk_i\}_{i \in \mathcal{C}})$ to $\mathcal{A}$.
2. For all oracle queries except $\text{OS}_2(\cdot)$, simulate identically to Hybrid_0 .
3. For $\text{OS}_2(\cdot)$ queries, simulate identically to Hybrid_0 except:
 - If $i \in \mathcal{C}_{\text{act}}$, compute $p_{i,j}$ and $p_{j,i}$ as in Hybrid_1 .
 - If $i \in \mathcal{H}_{\text{act}}$, query the PRF oracle with inputs $(i * n + j, \text{act} \parallel \text{msg} \parallel \text{pm}_1)$ and $(j * n + i, \text{act} \parallel \text{msg} \parallel \text{pm}_1)$, obtaining outputs y and y' . Set $p_{i,j} = y$, $p_{j,i} = y'$.
4. When $\mathcal{A}$ outputs a bit b_0 , output b_0 .

If PRF uses $\mathcal{O}_{\text{prf}}(\cdot)$, $\mathcal{B}$ perfectly simulates Hybrid_0 . Thus

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_0}(\lambda) = \Pr[1 \leftarrow \mathcal{B}^{\mathcal{O}_{\text{prf}}(\cdot)}(1^\lambda) : \text{seed}_i \leftarrow \{0, 1\}^\lambda].$$

If PRF uses $\mathcal{O}_{\text{rf}}(\cdot)$, under the condition that *the event bad₁ does not occur*, $\mathcal{B}$ perfectly simulates Hybrid_1 . Thus

$$\begin{aligned} \Pr[1 \leftarrow \mathcal{B}^{\mathcal{O}_{\text{rf}}(\cdot)}(1^\lambda) : \text{seed}_i \leftarrow \{0, 1\}^\lambda] &= \Pr[1 \leftarrow \mathcal{B}^{\mathcal{O}_{\text{rf}}(\cdot)}(1^\lambda) | \text{bad}_1] \cdot \Pr[\text{bad}_1] \\ &\quad + \Pr[1 \leftarrow \mathcal{B}^{\mathcal{O}_{\text{rf}}(\cdot)}(1^\lambda) | \neg \text{bad}_1] \cdot \Pr[\neg \text{bad}_1] \\ &\leq 2^{-N+1} + \text{Adv}_{\mathcal{A}}^{\text{hybrid}_1}(\lambda). \end{aligned}$$

Finally, we have

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{hybrid}_0}(\lambda) - \text{Adv}_{\mathcal{A}}^{\text{hybrid}_1}(\lambda) &= 2^{-N+1} + \Pr[1 \leftarrow \mathcal{B}^{\mathcal{O}_{\text{prf}}(\cdot)}(1^\lambda) : \text{seed}_i \leftarrow \{0, 1\}^\lambda] \\ &\quad - \Pr[1 \leftarrow \mathcal{B}^{\mathcal{O}_{\text{rf}}(\cdot)}(1^\lambda) : \text{seed}_i \leftarrow \{0, 1\}^\lambda] \leq \text{Adv}^{\text{prf}}(\lambda) + 2^{-N+1}. \end{aligned}$$

□

Hybrid₂. As illustrated in Figure 4b, the challenger samples an aggregated mask m_j at random directly, rather than sampling individual $p_{i,j}$ values for honest i, j separately. We assert that Hybrid_2 is identical to Hybrid_1 .

1. If $\text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] \neq \mathcal{H} \cap \text{act}$: There exists at least one honest signer $i_0 \notin \text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1]$. In Hybrid_1 , $p_{i_0,j}, p_{j,i_0} \leftarrow \mathcal{R}_q$; In Hybrid_2 , $m_j \leftarrow \mathcal{R}_q$. Because in Hybrid_1 , m_j includes a uniformly random term:

$$m_j = \sum_{i \in \text{act} \setminus \{i_0\}} (p_{i,j} - p_{j,i}) + (p_{i_0,j} - p_{j,i_0}),$$

hence these distributions are identical.

2. If $\text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] = \mathcal{H} \cap \text{act}$: This case corresponds to j being the last honest party, and also implies that all honest signers share a common view of the input $\text{act} \parallel \text{msg} \parallel \text{pm}_1$. In Hybrid_1 , all $p_{i,j}, p_{j,i}$ (for honest i, j) are fixed by prior interactions. In Hybrid_2 , the mask m_j is computed deterministically as:

$$m_j = - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} m_k.$$

We see that for $k \in \text{act}$,

$$\sum_{k \in \text{act}} m_k := \sum_{k \in \text{act}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) = 0.$$

Thus, these distributions are the same, because in Hybrid_1 :

$$\begin{aligned} m_j &= \sum_{k \in \mathcal{H}_{\text{act}}} m_k - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} m_k \\ &= \sum_{k \in \mathcal{H}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} m_k \\ &= - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} m_k. \end{aligned}$$

Hence, the two distributions are identical and

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_1}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{hybrid}_2}(\lambda)$$

Hybrid₃. As shown in Figure 5, this hybrid introduces three changes relative to Hybrid_2 :

1. Modified Hash Oracle H_1 : With probability ϕ , the challenger generates a trapdoor $\mathbf{B}_{f,g}$ by running $\text{Falcon.GenTrap}(1^\lambda)$ described in Theorem 2. The trapdoor $\mathbf{B}_{f,g}$ is then stored in the trapdoor table TDT. Finally, the challenger samples $h \leftarrow \mathcal{R}_q$ and outputs h .
2. Modified Signing Oracle OS_1 : After computing $h := \text{H}_1(\text{msg} \parallel \text{vk})$, the challenger checks the trapdoor table. If $\text{TDT}[\text{msg} \parallel \text{vk}] = \perp$, abort the query and return $\perp$.
3. Modified Verifying Condition: When adversary outputs a forgery $(\text{msg}^*, \text{sig}^*)$, the challenger checks the trapdoor table. If $\text{TDT}[\text{msg}^* \parallel \text{vk}] \neq \perp$, abort the game and return 0.

The Hybrid_3 successfully outputs 1 iff:

- Hybrid_2 outputs 1.
- All H_1 queries in OS_1 generated a trapdoor.
- The H_1 query for the forgery $(\text{msg}^*, \text{vk})$ did not generate a trapdoor.

The above three events are independent. Thus, the advantage difference between Hybrid_2 and Hybrid_3 is as follows:

$$\phi^{Q_S} (1 - \phi) \cdot \text{Adv}_{\mathcal{A}}^{\text{hybrid}_2}(\lambda) \leq \text{Adv}_{\mathcal{A}}^{\text{hybrid}_3}(\lambda),$$

where Q_S denotes the maximum number of signing oracle queries. By setting $\phi = Q_S / (Q_S + 1)$, we get

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_2}(\lambda) \leq e(Q_S + 1) \cdot \text{Adv}_{\mathcal{A}}^{\text{hybrid}_3}(\lambda).$$

Hybrid₄. This hybrid incorporates Falcon-type trapdoors as specified in Theorem 2. We give an illustration in Figure 6.

Hybrid₃	
<pre> 1 : ST, HT1, HT2 := ∅ 2 : Signed[·], TDT[·] := ⊥ 3 : C ← A(1^λ) 4 : if C ⊄ [n] or C ≥ t, return 0 5 : H := [n] \ C 6 : (vk, {sk_i}_{i∈[n]}) ← KGen(1^λ) 7 : (msg*, sig*) ← A^{H_i, OS_i}(vk, {sk_i}_{i∈C}) 8 : if msg* ∈ ST or Vf(vk, msg*, sig*) = 0 or TDT[msg* vk] ≠ ⊥ 9 : return 0 10 : else return 1 </pre>	
OS₁(j, msg)	H₁(str)
<pre> 1 : if j ∉ H, return ⊥ 2 : ST := ST ∪ {msg} 3 : (y_j, e'_j) ← D²_{σ₂}, (r_{j,1}, r_{j,2}) ← D²_{σ_r} 4 : h := H₁(msg vk) 5 : if TDT[msg vk] = ⊥, return ⊥ 6 : w_j := ay_j + e'_j + r_{j,1} + hr_{j,2} 7 : st_j := st_j ∪ {(msg, w_j, y_j, r_{j,2})} 8 : return pm₁[j] = w_j </pre>	<pre> 1 : if ∃ h s.t. (str, h) ∈ HT1, return h 2 : for probability = ϕ do 3 : (*, B_{f,g}) ← Falcon.GenTrap(1^λ) 4 : TDT[str] := B_{f,g} 5 : h ← R_q 6 : HT₁ := HT₁ ∪ {(str, h)} 7 : return h </pre>
OS₂(j, act, msg, pm₁)	H₂(str)
// Identical to the Hybrid ₂	// Identical to the Hybrid ₂

Fig. 5: Hybrid₃

Hybrid ₄	H ₁ (str)
// Identical to the Hybrid ₃ OS ₁ (j, msg) 1 : if $j \notin \mathcal{H}$, return $\perp$ 2 : $\text{ST} := \text{ST} \cup \{\text{msg}\}$ 3 : $(y_j, e'_j) \leftarrow D_{\sigma_2}^2$ 4 : $h := \text{H}_1(\text{msg} \parallel \text{vk})$ 5 : $\text{TDt}[\text{msg} \parallel \text{vk}] = \perp$, return $\perp$ 6 : $w_j \leftarrow \mathcal{R}_q$ 7 : $\text{st}_j := \text{st}_j \cup \{(\text{msg}, w_j, y_j, r_{j,2})\}$ 8 : return $\text{pm}_1[j] = w_j$	1 : if $\exists h$ s.t. $(\text{str}, h) \in \text{HT}_1$, return h 2 : for probability $= \phi$ do 3 : $(h, \mathbf{B}_{f,g}) \leftarrow \text{Falcon.GenTrap}(1^\lambda)$ 4 : $\text{TDt}[\text{str}] := \mathbf{B}_{f,g}$ 5 : else $h \leftarrow \mathcal{R}_q$ 6 : $\text{HT}_1 := \text{HT}_1 \cup \{(\text{str}, h)\}$ 7 : return h
OS ₂ (j, act, msg, pm ₁) 1 : if $j \notin \mathcal{H} \cap \text{act}$, return $\perp$ 2 : if $ \text{act} < t$ or $\text{act} \not\subseteq [n]$, return $\perp$ 3 : for $i \in \text{act}$ do 4 : $w_i := \text{pm}_1[i]$ 5 : if $\nexists (\text{msg}, w_j, *) \in \text{st}_j$, return $\perp$ 6 : Fetch $(\text{msg}, w_j, y_j, r_{j,2})$ from st_j 7 : if $j \in \text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1]$, return $\perp$ 8 : $\text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] := \text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] \cup \{j\}$ 9 : $w := \lfloor \sum_{i \in \text{act}} w_i \rfloor_{p_2}$ 10 : $c := \text{H}_2(w \parallel \text{msg} \parallel \text{vk})$ 11 : for $i \in \mathcal{C} \cap \text{act}$ do 12 : $p_{i,j} := \text{PRF}(\text{seed}_{i,j}, \text{act} \parallel \text{msg} \parallel \text{pm}_1)$ 13 : $p_{j,i} := \text{PRF}(\text{seed}_{j,i}, \text{act} \parallel \text{msg} \parallel \text{pm}_1)$ 14 : if $\text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] \neq \mathcal{H} \cap \text{act}$ 15 : $m_j \leftarrow \mathcal{R}_q$ 16 : else 17 : $m_j := - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} m_k$ 18 : $z_j := y_j + \lambda_{\text{act},j} s_j c + m_j$ 19 : $h := \text{H}_1(\text{msg} \parallel \text{vk})$ 20 : $\mathbf{B}_{f,g} := \text{TDt}[\text{msg} \parallel \text{vk}]$ 21 : $(r_{j,1}, r_{j,2}) \leftarrow \text{Falcon.SamplePre}(h, \mathbf{B}_{f,g}, w_j - ay_j - e'_j)$ 22 : $\text{st}_j := \text{st}_j \setminus \{(\text{msg}, w_j, y_j, r_{j,2})\}$ 23 : return $\text{pm}_2[j] = (z_j, r_{j,2})$	H ₂ (str) // Identical to the Hybrid ₀

Fig. 6: Hybrid₄

1. Modified Hash Oracle H_1 : With probability ϕ , the challenger generates a commitment key h and a trapdoor $\mathbf{B}_{f,g}$ by running $\text{Falcon.GenTrap}(1^\lambda)$. The trapdoor $\mathbf{B}_{f,g}$ is then stored in the trapdoor table TDT. Otherwise (with probability $1 - \phi$), the challenger samples $h \leftarrow \mathcal{R}_q$. The distinction from Hybrid_3 is that the commitment key h may be sampled from $\text{Falcon.GenTrap}(1^\lambda)$ rather than a uniform distribution.
2. Modified Signing Oracle OS_1 : The challenger samples $w_j \leftarrow \mathcal{R}_q$ directly, without sampling $(r_{j,1}, r_{j,2})$.
3. Modified Signing Oracle OS_2 : The challenger computes $(r_{j,1}, r_{j,2})$ via $\text{Falcon.SamplePre}(h, \mathbf{B}_{f,g}, w_j - ay_j - e'_j)$.

The property of Falcon.SamplePre guarantees computational indistinguishability between Hybrid_3 and Hybrid_4 . Furthermore,

$$\text{Adv}_{\mathcal{A}}^{\text{Hybrid}_3}(\lambda) - \text{Adv}_{\mathcal{A}}^{\text{Hybrid}_4}(\lambda) \leq Q_S \cdot \text{Adv}_{\text{ind}}^{\text{falcon}}(\lambda).$$

Hybrid₅. As shown in Figure 7, this hybrid changes analogously to Hybrid_2 . In OS_2 , the challenger samples the response z_j directly either at random or consistently for the last honest party. We assert that Hybrid_5 is identical to Hybrid_4 .

1. When $\text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] \neq \mathcal{H} \cap \text{act}$: In Hybrid_4 , $m_j \leftarrow \mathcal{R}_q$, implying $z_j = y_j + \lambda_{\text{act},j} s_j c + m_j$ is uniformly random over $\mathcal{R}_q$. Thus, it is distributed identically as $z_j \leftarrow \mathcal{R}_q$ in Hybrid_5 .
2. When $\text{Signed}[\text{act} \parallel \text{msg} \parallel \text{pm}_1] = \mathcal{H} \cap \text{act}$: We observe that all honest signers engage in the second-round interaction with the same input $\text{act} \parallel \text{msg} \parallel \text{pm}_1$, fixing the common challenge $c = H_2(w \parallel \text{msg} \parallel \text{vk})$. Under this condition, the response z_j in Hybrid_4 can be rewritten as:

$$\begin{aligned} z_j &= \sum_{k \in \mathcal{H}_{\text{act}}} z_k - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} z_k \\ &= \sum_{k \in \mathcal{H}_{\text{act}}} (y_k + \lambda_{\text{act},k} s_k c + m_k) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} z_k \\ &= \sum_{k \in \mathcal{H}_{\text{act}}} y_k + (s - \hat{s})c + \sum_{k \in \mathcal{H}_{\text{act}}} m_k - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} z_k, \end{aligned}$$

where $\hat{s} := \sum_{k \in \mathcal{C}_{\text{act}}} \lambda_{\text{act},k} s_k$. The sum of honest parties' masks satisfies:

$$\sum_{k \in \mathcal{H}_{\text{act}}} m_k = - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}),$$

hence the distribution of z_j from Hybrid_4 is identical to Hybrid_5 and we have

$$\text{Adv}_{\mathcal{A}}^{\text{Hybrid}_4}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{Hybrid}_5}(\lambda).$$

Hybrid₆. In this hybrid (Figure 8), the first-round ephemeral randomness (y_j, e'_j) are no longer sampled. Instead, they are generated during OS_2 . This modification preserves indistinguishability from Hybrid_5 .

Hybrid ₅	H ₁ (str)
// Identical to the Hybrid ₃	// Identical to the Hybrid ₄
OS ₁ (j, msg)	H ₂ (str)
// Identical to the Hybrid ₄	// Identical to the Hybrid ₀
OS ₂ (j, act, msg, pm ₁)	
<pre> 1 : if j ∉ H ∩ act, return ⊥ 2 : if act < t or act ⊈ [n], return ⊥ 3 : for i ∈ act do 4 : w_i := pm₁[i] 5 : if ∄(msg, w_j, *) ∈ st_j, return ⊥ 6 : Fetch (msg, w_j, y_j, r_{j,2}) from st_j 7 : if j ∈ Signed[act msg pm₁], return ⊥ 8 : Signed[act msg pm₁] := Signed[act msg pm₁] ∪ {j} 9 : w := ⌊ ∑_{i ∈ act} w_i ⌋_{p₂} 10 : c := H₂(w msg vk) 11 : for i ∈ C ∩ act do 12 : p_{i,j} := PRF(seed_{i,j}, act msg pm₁) 13 : p_{j,i} := PRF(seed_{j,i}, act msg pm₁) 14 : if Signed[act msg pm₁] ≠ H ∩ act 15 : z_j ← R_q 16 : else 17 : z_j := ∑_{k ∈ H_{act}} y_k + (s - ŝ)c - ∑_{k ∈ C_{act}} ∑_{i ∈ act} (p_{i,k} - p_{k,i}) - ∑_{k ∈ H_{act} \ {j}} z_k 18 : h := H₁(msg vk) 19 : B_{f,g} := TDT[msg vk] 20 : (r_{j,1}, r_{j,2}) ← Falcon.SamplePre(h, B_{f,g}, w_j - ay_j - e'_j) 21 : st_j := st_j \ {(msg, w_j, y_j, r_{j,2})} 22 : return pm₂[j] = (z_j, r_{j,2}) </pre>	

Fig. 7: Hybrid₅

Hybrid ₆	H ₁ (str)
// Identical to the Hybrid ₃	// Identical to the Hybrid ₄
OS ₁ (j, msg)	H ₂ (str)
// Identical to the Hybrid ₄	// Identical to the Hybrid ₀
OS ₂ (j, act, msg, pm ₁)	
<pre> 1 : if j ∉ H ∩ act, return ⊥ 2 : if act < t or act ⊈ [n], return ⊥ 3 : for i ∈ act do 4 : w_i := pm₁[i] 5 : if ∄(msg, w_j, *) ∈ st_j, return ⊥ 6 : Fetch (msg, w_j, y_j, r_{j,2}) from st_j 7 : if j ∈ Signed[act msg pm₁], return ⊥ 8 : Signed[act msg pm₁] := Signed[act msg pm₁] ∪ {j} 9 : w := ⌊ ∑_{i ∈ act} w_i ⌋_{p₂} 10 : c := H₂(w msg vk) 11 : for i ∈ C ∩ act do 12 : p_{i,j} := PRF(seed_{i,j}, act msg pm₁) 13 : p_{j,i} := PRF(seed_{j,i}, act msg pm₁) 14 : (y_j, e'_j) ← D²_{σ₂} 15 : if Signed[act msg pm₁] ≠ H ∩ act 16 : z_j ← R_q 17 : ŵ_j = w_j - ay_j - e'_j 18 : else 19 : (y_{sid}, e_{sid}) := (y_j + sc, e'_j + ec) 20 : ŵ_j := w_j - ay_{sid} + b̂c - e_{sid} 21 : z_j := y_{sid} + ∑_{k ∈ H_{act} \ {j}} (y_k - z_k) - ŝc - ∑_{k ∈ C_{act}} ∑_{i ∈ act} (p_{i,k} - p_{k,i}) 22 : h := H₁(msg vk) 23 : B_{f,g} := TDT[msg vk] 24 : (r_{j,1}, r_{j,2}) ← Falcon.SamplePre(h, B_{f,g}, ŵ_j) 25 : st_j := st_j \ {(msg, w_j, y_j, r_{j,2})} 26 : return pm₂[j] = (z_j, r_{j,2}) </pre>	

Fig. 8: Hybrid₆

1. When $\text{Signed}[\text{act}||\text{msg}||\text{pm}_1] \neq \mathcal{H} \cap \text{act}$: The only difference is that (y_j, e'_j) is deferred to OS_2 for sampling, which does not affect the generation of any values.
2. When $\text{Signed}[\text{act}||\text{msg}||\text{pm}_1] = \mathcal{H} \cap \text{act}$: The computations of $\hat{w}_j$ and z_j remains equivalent to the Hybrid_5 . In Hybrid_5 :

$$\hat{w}_j = w_j - a(y_{\text{sid}} - sc) - (e_{\text{sid}} - ec) = w_j - ay_{\text{sid}} + \hat{b}c - e_{\text{sid}},$$

where $y_{\text{sid}} := y_j + sc$, $e_{\text{sid}} := e'_j + ec$, and $\hat{b} := as + e$ denotes the original public key. For the response z_j ,

$$\begin{aligned} z_j &= \sum_{k \in \mathcal{H}_{\text{act}}} y_k + (s - \hat{s})c - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}) - \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} z_k \\ &= y_{\text{sid}} + \sum_{k \in \mathcal{H}_{\text{act}} \setminus \{j\}} (y_k - z_k) - \hat{s}c - \sum_{k \in \mathcal{C}_{\text{act}}} \sum_{i \in \text{act}} (p_{i,k} - p_{k,i}). \end{aligned}$$

Hence, we have

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_5}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{hybrid}_6}(\lambda).$$

Hybrid₇. This hybrid is the same as Hybrid_6 , except that in the key generation phase, the original public key $\hat{b}$ is uniformly sampled rather than being computed as $as + e$, and s_i are secret shares of 0 instead of s . We observe that in Hybrid_6 , the challenger simulates oracles without using any secret share of honest signers. So the modification of the secret sharing is indistinguishable. Meanwhile, the challenger discards usage of the secret (s, e) , except for $(y_{\text{sid}}, e_{\text{sid}})$ in OS_2 . Under the Hint-RLWE assumption, we assert that Hybrid_6 and Hybrid_7 are computationally indistinguishable via the following [Lemma 7](#).

Lemma 7. *If there exists a PPT adversary $\mathcal{A}$ that distinguishes Hybrid_6 from Hybrid_7 , then a PPT adversary $\mathcal{B}$ can be constructed to break the **Hint-RLWE** assumption. Furthermore:*

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_6}(\lambda) - \text{Adv}_{\mathcal{A}}^{\text{hybrid}_7}(\lambda) \leq \text{Adv}_{\mathcal{R}, q, 1, \sigma_1, \sigma_2, \tau, Q_H}^{\text{hint-rlwe}}(\lambda).$$

Proof. We construct the reduction $\mathcal{B}$ as follows:

1. Given an instance $(a^*, b^*, \{c_i^*, \text{ht}_i^*\}_{i \in [Q_H]})$, set $a := a^*$, $\hat{b} := b^*$. Compute all other parameters as in Hybrid_7 and provide $(\text{vk}, \{\text{sk}_i\}_{i \in \mathcal{C}})$ to $\mathcal{A}$.
2. For all oracle queries except $\text{H}_2(\cdot)$ and $\text{OS}_2(\cdot)$, simulate identically to Hybrid_7 .
3. For the i -th $\text{H}_2(\cdot)$ queries, if $\text{H}_2(\cdot)$ is unprogrammed, set $c_i := c_i^*$.
4. For $\text{OS}_2(\cdot)$ queries:
 - If $\text{Signed}[\text{act}||\text{msg}||\text{pm}_1] \neq \mathcal{H} \cap \text{act}$, simulate identically to Hybrid_7 .
 - If $\text{Signed}[\text{act}||\text{msg}||\text{pm}_1] = \mathcal{H} \cap \text{act}$, calculate $c_{\text{sid}} := \text{H}_2(w||\text{msg}||\text{vk})$. Then look up $c_{\text{sid}} = c_i$ and the corresponding $(c_i, \text{ht}_i^* = (ht_{i,1}^*, ht_{i,2}^*))$ from the instance. Set $y_{\text{sid}} := ht_{i,1}^*$, $e_{\text{sid}} := ht_{i,2}^*$.
5. When $\mathcal{A}$ outputs a bit b_0 , output b_0 .

Hybrid ₇	
1 : $\text{ST}, \text{HT1}, \text{HT2} := \emptyset$	
2 : $\text{Signed}[\cdot], \text{TDT}[\cdot] := \perp$	
3 : $\mathcal{C} \leftarrow \mathcal{A}(1^\lambda)$	
4 : if $\mathcal{C} \not\subseteq [n]$ or $ \mathcal{C} \geq t$, return 0	
5 : $\mathcal{H} := [n] \setminus \mathcal{C}$	
6 : $(\text{vk}, \{\text{sk}_i\}_{i \in [n]}) \leftarrow \widehat{\text{KGen}}(1^\lambda)$	
7 : $(\text{msg}^*, \text{sig}^*) \leftarrow \mathcal{A}^{\text{H}_i, \text{OS}_i}(\text{vk}, \{\text{sk}_i\}_{i \in \mathcal{C}})$	
8 : if $\text{msg}^* \in \text{ST}$ or $\text{Vf}(\text{vk}, \text{msg}^*, \text{sig}^*) = 0$ or $\text{TDT}[\text{msg}^* \text{vk}] \neq \perp$	
9 : return 0	
10 : else return 1	
$\text{OS}_1(j, \text{msg})$	$\text{H}_1(\text{str})$
// Identical to the Hybrid ₄	// Identical to the Hybrid ₄
$\text{OS}_2(j, \text{act}, \text{msg}, \text{pm}_1)$	$\text{H}_2(\text{str})$
// Identical to the Hybrid ₆	// Identical to the Hybrid ₀

Fig. 9: Hybrid₇

$\widehat{\text{KGen}}(1^\lambda, t, n)$
1 : $(a, \hat{b}) \leftarrow \mathcal{R}_q^2$, $(s, e) \leftarrow D_{\sigma_1}^2$
2 : $b := \lfloor \hat{b} \rfloor_{p_1}$
3 : $\{s_i\}_{i \in [n]} \leftarrow \text{SS.Share}(0)$
4 : for $i, j = 1$ to n do
5 : $\text{seed}_{i,j} \leftarrow \{0, 1\}^\lambda$
6 : for $i \in \mathcal{C}$ do
7 : $\text{sk}_i := (s_i, (\text{seed}_{i,j}, \text{seed}_{j,i})_{j \in [n]})$
8 : for $i \in \mathcal{H}$ do
9 : $\text{sk}_i := \perp$
10 : $\text{vk} := (a, b)$
11 : return $(\text{vk}, (\text{sk}_i)_{i \in [n]})$

Fig. 10: Hybrid₇- $\widehat{\text{KGen}}$

Hybrid₇ is the same as Hybrid₆, except generation of $\hat{b}$, and one bad event bad_2 : a single c_{sid} is used in multiple sessions. Under the condition that *the event bad₂ does not occur*, when b^* is an RLWE instance, $\mathcal{B}$ perfectly simulates Hybrid₆. Otherwise, $\mathcal{B}$ perfectly simulates Hybrid₇. Following TRaccoon's entropy argument [dPKM⁺24, lemma 3.8], with probability $1 - 2^{-N}$ over the choice of a , $H_\infty([\sum_i w_i]_{p_2}) \geq N$. Thus, the probability that c_{sid} is reused is bounded by 2^{-N+1} , that is $\Pr[\text{bad}_2] \leq 2^{-N+1}$. Overall, we have

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_6}(\lambda) - \text{Adv}_{\mathcal{A}}^{\text{hybrid}_7}(\lambda) \leq \text{Adv}_{\mathcal{R}, q, 1, \sigma_1, \sigma_2, \tau, Q_H}^{\text{hint-rlwe}}(\lambda) + 2^{-N+1}.$$

Finally, we bound the advantage of Hybrid₇. Our proof relies on the forking lemma [BN06]. To construct a reduction that breaks RSIS, we have to make the adversary submit two valid openings for a single commitment under the same key.

Lemma 8. *For any PPT adversary $\mathcal{A}$ against Hybrid₇,*

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_7}(\lambda) \leq \frac{Q_H^2}{|\mathcal{C}|} + Q_H^{3/2} \cdot \sqrt{\text{Adv}_{\mathcal{R}, q, 4, \beta_{\text{SIS}}}^{\text{rsis}}(\lambda)},$$

$$\text{where } \beta_{\text{SIS}} := \sqrt{4\beta_z^2 + 4\beta_r^2 + 4\tau + \beta_\delta^2}, \beta_\delta := \frac{q}{q_2}(\sqrt{N} + 2\beta_\Delta) + \frac{q\tau}{q_1}\sqrt{N}.$$

Proof. We construct the reduction $\mathcal{B}$ against $\mathbf{RSIS}_{\mathcal{R}, q, 4, \beta_{\text{SIS}}}$ as follows:

1. Given an instance $\mathbf{a}^* = (a_1^*, a_2^*, a_3^*)$, set $a := a_1^*$ and $\hat{b} := a_2^*$. Compute all other parameters as in Hybrid₇ and provide $(\text{vk}, \{\text{sk}_i\}_{i \in \mathcal{C}})$ to $\mathcal{A}$.
2. Sample $i^* \leftarrow [Q_H]$. For the i^* -th $H_1(\cdot)$ queries, if $H_1(\cdot)$ is unprogrammed, return $h_{i^*} := a_3^*$.
3. On forgery $(\text{msg}^*, (z^*, c^*, r^*, \Delta^*))$ from $\mathcal{A}$, compute $h^* := H_1(\text{msg}^* \parallel \text{vk})$ and $w^* := \Delta^* + \lfloor az^* - \frac{q}{q_1}bc^* + h^*r^* \rfloor_{p_2}$. If $h^* \neq h_{i^*}$ or the forgery is invalid, abort the game and output $\perp$.
4. After applying the forking lemma, $\mathcal{A}$ submits the second forgery $(\text{msg}', (z', c', r', \Delta'))$. Compute $h' := H_1(\text{msg}' \parallel \text{vk})$ and $w' := \Delta' + \lfloor az' - \frac{q}{q_1}bc' + h'r' \rfloor_{p_2}$. Return $\perp$ if the second forgery is invalid.
5. If $\text{msg}^* = \text{msg}'$ and $w^* = w'$, then $h^* = h'$ and the reduction $\mathcal{B}$ can output an RSIS solution $(\delta, z^* - z', -c^* + c', r^* - r')$ under the bound $\beta_{\text{SIS}} := \sqrt{4\beta_z^2 + 4\beta_r^2 + 4\tau + \beta_\delta^2}$ s.t.

$$[1 \ a \ \hat{b} \ h^*] \cdot \begin{bmatrix} \delta \\ z^* - z' \\ -c^* + c' \\ r^* - r' \end{bmatrix} = 0.$$

Otherwise, output $\perp$.

We first verify the boundary condition. Firstly, we have

$$\|(z^* - z', -c^* + c', r^* - r')\| \leq \sqrt{4\beta_z^2 + 4\beta_r^2 + 4\tau}.$$

Then,

$$\begin{aligned}
 \|\delta\| &= \|a(z^* - z') - \hat{b}(c^* - c') + h^*(r^* - r')\| \\
 &\leq \|(az^* - \frac{q}{q_1}bc^* + h^*r^*) - (az' - \frac{q}{q_1}bc' + h^*r')\| + \|(\frac{q}{q_1}b - \hat{b})(c^* - c')\|. \\
 &\leq \frac{q}{q_2}(\|\mathbf{1}\| + \|\Delta^* - \Delta'\|) + \frac{q}{2q_1}\sqrt{N} \cdot 2\tau \leq \beta_\delta
 \end{aligned}$$

Thus, we have the bound for the solution $(\delta, z^* - z', -c^* + c', r^* - r')$ is $\beta_{SIS} := \sqrt{4\beta_z^2 + 4\beta_r^2 + 4\tau + \beta_\delta^2}$.

We then analyze the success probability of $\mathcal{B}$. Suppose the following probabilities:

– Forging probability:

$$p_{\text{forgery}} := \Pr[h^* = a_3^* \ \& \ \text{Verify}(\text{vk}, \text{msg}^*, (z^*, c^*, r^*, \Delta^*)) = 1].$$

– Forking probability:

$$p_{\text{forking}} := \Pr[\mathcal{B} \text{ successfully outputs an RSIS solution}].$$

Under the construction of Hybrid_7 , we have

$$p_{\text{forgery}} \geq \frac{1}{Q_H} \text{Adv}_{\mathcal{A}}^{\text{hybrid}_7}(\lambda).$$

Under the forking lemma, we have

$$p_{\text{forgery}} \leq \frac{Q_H}{|\mathcal{C}|} + \sqrt{Q_H \cdot p_{\text{forking}}}.$$

Under $\mathbf{RSIS}_{\mathcal{R}, q, 4, \beta_{SIS}}$ assumption, we have

$$p_{\text{forking}} \leq \text{Adv}_{\mathcal{R}, q, 4, \beta_{SIS}}^{\text{rsis}}(\lambda).$$

We conclude that

$$\text{Adv}_{\mathcal{A}}^{\text{hybrid}_7}(\lambda) \leq \frac{Q_H^2}{|\mathcal{C}|} + Q_H^{3/2} \cdot \sqrt{\text{Adv}_{\mathcal{R}, q, 4, \beta_{SIS}}^{\text{rsis}}(\lambda)}.$$

Finally, by combining all results, we obtain:

$$\begin{aligned}
 \text{Adv}_{\mathcal{A}}^{\text{hybrid}_0}(\lambda) &\leq \text{Adv}^{\text{prf}}(\lambda) + \frac{1}{2^{N-1}} + e(Q_S + 1) \cdot \left[Q_S \cdot \text{Adv}_{\text{ind}}^{\text{falcon}}(\lambda) + \right. \\
 &\quad \left. \text{Adv}_{\mathcal{R}, q, 1, \sigma_1, \sigma_2, \tau, Q_H}^{\text{hint-rlwe}}(\lambda) + \frac{1}{2^{N-1}} + \frac{Q_H^2}{|\mathcal{C}|} + Q_H^{3/2} \cdot \sqrt{\text{Adv}_{\mathcal{R}, q, 4, \beta_{SIS}}^{\text{rsis}}(\lambda)} \right].
 \end{aligned}$$